
PROYECTO FINAL INTEGRADOR

30 DE MAYO DE 2026
UNIVERSIDAD ANTONIO NARIÑO

PROYECTO FINAL INTEGRADOR

Luis Carlos Rojas Criollo

**Universidad Antonio Nariño
FACULTAD DE SISTEMAS**

**Desarrollo de Aplicaciones Seguras
ESPECIALIZACIÓN EN CIBERSEGURIDAD
Mayo 2026.**

Tabla de contenido

Introducción	4
Objetivo	4
Alcance del Proyecto	4
Contexto de la Aplicación	4
Análisis de Requerimientos	5
Requerimientos Funcionales	5
Requerimientos no Funcionales	5
Requerimientos de Seguridad	6
Posibles vulnerabilidades y amenazas.....	7
Diseño Seguro de la Aplicación.....	7
Arquitectura Propuesta de la Aplicación.....	7
Módulos Principales del Sistema	9
Modelo de Datos Propuesto	9
Modelado de Amenazas con STRIDE	10
Controles de Seguridad Definidos	11
Desarrollo de Código en Laravel	12
Implementación De Autenticación con Laravel Breeze	15
Creación de Roles de Usuario.....	19
Creación de Módulos Principales	23
Prueba de Funcionamiento	25
Movimientos de Inventario	26
Manejo de errores, logs y Monitoreo Básico	31
Páginas Personalizadas De Error.....	32
Prueba de Monitoreo	33
Pruebas	33
Entorno de Pruebas	33
Casos de Prueba Funcionales	34
Casos de Prueba de Autorización y Roles	35
Pruebas de Validación de Datos	35
Pruebas de Seguridad Básicas	36
Pruebas específicas	37
Prueba Específica de SQL Injection	37
Prueba Específica de XSS	37
Prueba de salida mayor al stock.....	38
Prueba de Manejo Seguro de Errores	39
Manejo de Incidentes.....	39

Plan de Respuesta a Incidentes	40
Registro de Incidentes en GitHub Issues	40
Labels en GitHub	41
Workflow de Gestión de Incidentes.....	41
Bitácora de Incidentes y Soluciones	42
Monitoreo y Detección de Amenazas	42
Procedimiento para Manejar Vulnerabilidades y Exploits	43
Métricas de Incidentes.....	44
Proyecto en GitHub.....	44
Conclusión.....	45
Resultado de aprendizaje.....	45
Referencias	46

Introducción

El presente informe documenta el desarrollo del proyecto final integrador de la materia Desarrollo de Aplicaciones Seguras, el cual consiste en la construcción de una aplicación web segura para la gestión de inventarios de la empresa Distribuidora S.A.

La aplicación se desarrolla aplicando conceptos vistos durante el semestre como ciclo de desarrollo seguro, autenticación, autorización por roles, validación de entradas, manejo de errores, registro de eventos, monitoreo, pruebas de seguridad y gestión de incidentes.

Durante el desarrollo se implementan módulos para la gestión de categorías, proveedores, productos y movimientos de inventario. También, se incorporaron controles de seguridad para proteger el acceso al sistema, evitar modificaciones no autorizadas, prevenir errores de integridad en el stock y mantener trazabilidad sobre las acciones realizadas por los usuarios.

Objetivo

Implementar una aplicación web segura para la gestión de inventarios de Distribuidora S.A., aplicando buenas prácticas de desarrollo seguro de software, autenticación, autorización por roles, validación de datos, manejo de errores, monitoreo, pruebas y gestión de incidentes.

Alcance del Proyecto

El proyecto comprende el desarrollo de una aplicación web local para la gestión de inventarios, orientada a controlar productos, categorías, proveedores y movimientos de stock. El sistema permite registrar entradas, salidas y ajustes de inventario, manteniendo trazabilidad del usuario responsable, stock anterior y stock nuevo.

El alcance incluye autenticación de usuarios, control de acceso por roles, validación de formularios, manejo seguro de errores, registro de eventos mediante logs, panel básico de monitoreo, pruebas funcionales y de seguridad básica, además de un proceso de gestión de incidentes apoyado en GitHub.

No se incluye un despliegue en producción real, integración con sistemas externos ni facturación electrónica. El proyecto se desarrolla en un entorno local académico con fines de aprendizaje y demostración de buenas prácticas de desarrollo seguro.

Contexto de la Aplicación

Distribuidora S.A. es una empresa colombiana dedicada a la distribución de productos de consumo masivo. Actualmente requiere una aplicación web que permita gestionar de forma organizada el inventario de productos, controlar entradas y salidas, consultar existencias y mantener trazabilidad de los movimientos realizados por los usuarios.

La aplicación será desarrollada en Laravel, utilizando MySQL como base de datos y Visual Studio Code como entorno de desarrollo. La elección de Laravel se basa en que en el transcurso del semestre se trabajó previamente con este framework en la implementación de autenticación, roles, sesiones, middleware, validaciones y manejo básico de errores, lo cual permite aplicar conocimientos ya vistos en la materia de Desarrollo de Aplicaciones Seguras.

Análisis de Requerimientos

El análisis de requerimientos permite establecer una base clara para el desarrollo del sistema de inventarios. A partir de esta información se definen las funcionalidades principales, las condiciones de calidad esperadas y los controles de seguridad que deben aplicarse durante el proyecto, en concordancia con los objetivos del proyecto final de implementar una aplicación web segura.

Requerimientos Funcionales

A partir del contexto de Distribuidora S.A. y de la necesidad de controlar los productos, existencias y movimientos de inventario, se identifican los principales requerimientos funcionales de la aplicación. Estos requerimientos describen las acciones que el sistema debe permitir realizar a los usuarios según el objetivo del proyecto.

Código	Requerimiento funcional	Descripción
RF-01	Inicio de sesión	El sistema debe permitir que los usuarios autorizados ingresen mediante correo y contraseña.
RF-02	Gestión de usuarios	El administrador debe poder crear usuarios y asignarles un rol dentro del sistema.
RF-03	Gestión de productos	El sistema debe permitir registrar, consultar, actualizar y desactivar productos.
RF-04	Gestión de categorías	El sistema debe permitir clasificar los productos por categorías.
RF-05	Gestión de proveedores	El sistema debe permitir registrar proveedores asociados a los productos.
RF-06	Entradas de inventario	El sistema debe permitir registrar ingresos de productos al inventario.
RF-07	Salidas de inventario	El sistema debe permitir registrar salidas de productos del inventario.
RF-08	Consulta de stock	El sistema debe permitir consultar la cantidad disponible de cada producto.
RF-09	Historial de movimientos	El sistema debe guardar un registro de entradas, salidas y ajustes realizados.
RF-10	Alerta de stock bajo	El sistema debe mostrar productos con cantidad inferior al mínimo definido.

Requerimientos no Funcionales

Además de las funciones principales del sistema, es necesario definir condiciones de calidad, seguridad y operación que permitan que la aplicación sea confiable, mantenible y adecuada para el uso de la empresa. En la siguiente tabla se presentan los requerimientos no funcionales más importantes para el desarrollo seguro de la aplicación.

Código	Requerimiento no funcional	Descripción
RNF-01	Seguridad	El sistema debe proteger el acceso mediante autenticación y autorización por roles.
RNF-02	Confidencialidad	Solo los usuarios autorizados podrán consultar información del inventario y movimientos.
RNF-03	Integridad	El sistema debe validar los datos antes de registrarlos para evitar información incompleta o incorrecta.
RNF-04	Disponibilidad	La aplicación debe estar disponible en el entorno local de pruebas durante el desarrollo.
RNF-05	Trazabilidad	Las acciones importantes deben quedar registradas para facilitar auditoría y seguimiento.
RNF-06	Usabilidad	La interfaz debe ser sencilla para que el personal de operaciones pueda usarla sin dificultad.
RNF-07	Manejo de errores	Los errores deben mostrarse con mensajes claros, sin revelar detalles técnicos internos.
RNF-08	Mantenibilidad	El código debe estar organizado en rutas, controladores, modelos, vistas y migraciones propias de Laravel.

Requerimientos de Seguridad

Teniendo en cuenta que la aplicación manejará información relacionada con productos, usuarios, proveedores y movimientos de inventario, se deben establecer controles de seguridad desde la etapa de análisis. Estos requerimientos buscan proteger el acceso al sistema, la integridad de los datos y la trazabilidad de las operaciones realizadas.

Categoría	Requerimiento de seguridad
Autenticación	El sistema debe exigir inicio de sesión para acceder a los módulos internos.
Autorización	El acceso a las funciones debe depender del rol del usuario.
Confidencialidad	No se debe mostrar información interna del sistema ni errores técnicos al usuario final.
Integridad	Las entradas de datos deben validarse antes de guardarse en la base de datos.
Disponibilidad	El sistema debe manejar errores sin interrumpir completamente la operación.
Registro y monitoreo	Se deben registrar eventos relevantes como inicios de sesión, errores y movimientos de inventario.
Protección contra inyección	Las consultas a base de datos deben realizarse mediante Eloquent, evitando concatenar entradas del usuario.
Protección contra XSS	Los datos ingresados por usuarios deben validarse y mostrarse usando las protecciones propias de Blade.
Gestión de sesiones	La sesión debe cerrarse correctamente y proteger rutas internas.

Posibles vulnerabilidades y amenazas

Como parte del análisis inicial, también se identifican posibles amenazas y vulnerabilidades que podrían afectar la aplicación si no se aplican controles adecuados. Esta revisión permite anticipar riesgos comunes en aplicaciones web y relacionarlos con medidas preventivas que serán consideradas durante el diseño, desarrollo y pruebas del sistema.

Amenaza o vulnerabilidad	Impacto posible	Medida preventiva
Acceso no autorizado	Un usuario podría ver o modificar información sin permiso.	Implementar roles y middleware.
Inyección SQL	Manipulación de consultas o extracción de datos.	Usar validaciones y consultas parametrizadas.
XSS	Ejecución de scripts maliciosos en el navegador.	Validar entradas y escapar salidas en las vistas.
Contraseñas débiles	Riesgo de acceso indebido a cuentas.	Usar hash de contraseñas y reglas mínimas de seguridad.
Errores técnicos visibles	Exposición de rutas, código o detalles internos.	Mostrar mensajes genéricos y registrar detalles en logs.
Manipulación de stock	Alteración no autorizada de cantidades.	Restringir movimientos según rol y registrar historial.
Falta de trazabilidad	Dificultad para saber quién realizó un cambio.	Registrar usuario, fecha, tipo de movimiento y producto afectado.
Pérdida de disponibilidad	Interrupción del servicio ante errores.	Manejo adecuado de excepciones y pruebas previas.

Diseño Seguro de la Aplicación

El diseño seguro de la aplicación permite definir la estructura técnica y funcional del sistema antes de iniciar la programación. Para el sistema web de gestión de inventarios de Distribuidora S.A., se plantea una arquitectura basada en Laravel, MySQL y el patrón MVC, incorporando controles de seguridad desde el diseño para reducir riesgos asociados al acceso no autorizado, manipulación de datos, errores de validación y exposición de información sensible.

Durante esta etapa se definen los componentes principales de la aplicación, el flujo de información, el modelo de datos, las amenazas mediante STRIDE y los controles de seguridad que serán implementados posteriormente durante el desarrollo.

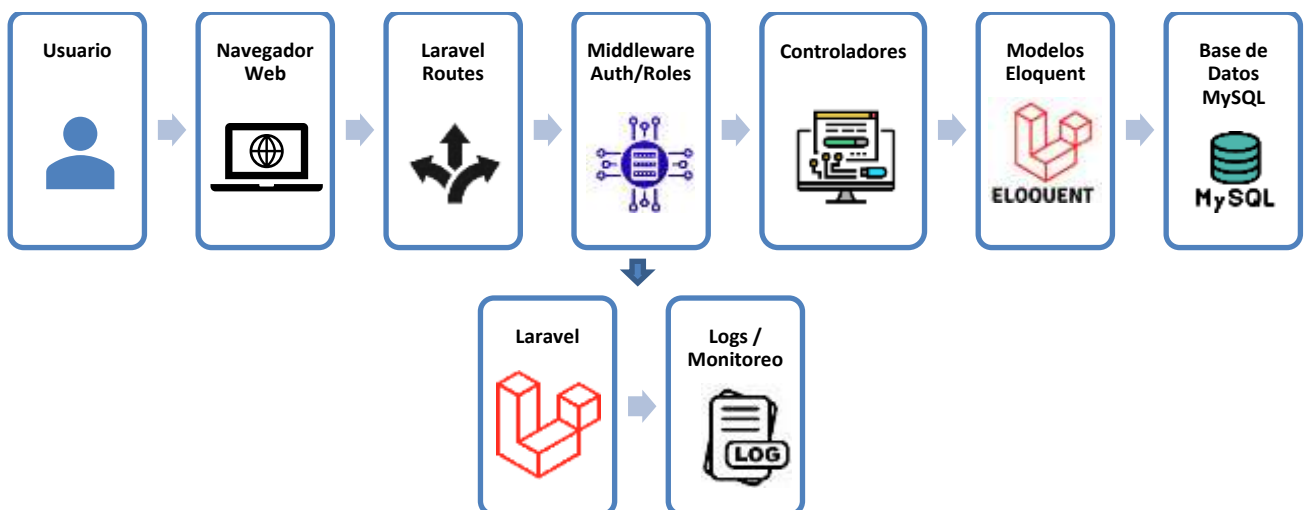
Arquitectura Propuesta de la Aplicación

La arquitectura propuesta se basa en una aplicación web desarrollada en Laravel, donde los usuarios interactúan desde el navegador con las vistas del sistema. Laravel se encarga de gestionar las rutas, controladores, modelos, validaciones, middleware de autenticación y autorización, mientras que MySQL almacena la información relacionada con usuarios, productos, proveedores, categorías y movimientos de inventario.

Componente	Descripción	Seguridad aplicada
Usuario final	Persona que accede al sistema desde el navegador. Puede ser administrador, operador o usuario de consulta.	Acceso controlado mediante usuario, contraseña y rol asignado.
Navegador web	Medio desde el cual se accede a la aplicación.	Uso de formularios protegidos, validación de datos y protección CSRF propia de Laravel.
Rutas Laravel	Definen las URL disponibles del sistema.	Rutas protegidas mediante middleware de autenticación y roles.
Middleware	Filtra solicitudes antes de llegar al controlador.	Restringe accesos no autorizados y aplica control por rol.
Controladores	Procesan las solicitudes del usuario y ejecutan la lógica principal.	Validan permisos y envían datos seguros hacia los modelos o vistas.
Modelos Laravel	Representan las entidades del sistema como Producto, Categoría, Proveedor y Movimiento.	Uso de Eloquent para evitar consultas SQL inseguras.
Base de datos MySQL	Almacena usuarios, productos, proveedores, categorías y movimientos.	Relaciones, validaciones y control de acceso desde la aplicación.
Logs del sistema	Registran eventos importantes y errores.	Apoyan la trazabilidad, monitoreo y respuesta ante incidentes.

Esta arquitectura permite separar responsabilidades entre presentación, lógica de negocio y persistencia de datos. Además, facilita la aplicación de controles de seguridad como autenticación, autorización, validación de entradas, registro de eventos y manejo controlado de errores.

En el siguiente diagrama se visualiza con mayor claridad el flujo general de la aplicación.



En resumen, el usuario accede desde el navegador, las solicitudes pasan por las rutas de Laravel y luego son filtradas por middleware de autenticación y roles. Si el usuario tiene permisos, la solicitud llega al controlador correspondiente, el cual valida los datos y utiliza los modelos de Laravel para consultar o modificar la información almacenada en MySQL. Los eventos relevantes y errores controlados se registran en logs para apoyar la trazabilidad y el monitoreo.

Módulos Principales del Sistema

Para organizar el desarrollo del sistema, se definen los módulos principales que tendrá la aplicación. Estos módulos responden a los requerimientos funcionales identificados en la etapa anterior y permiten dividir el sistema en partes más fáciles de desarrollar, probar y proteger.

Módulo	Función principal	Usuarios permitidos
Autenticación	Permite iniciar y cerrar sesión en el sistema.	Administrador, operador y consulta.
Gestión de usuarios	Permite crear usuarios y asignar roles.	Administrador.
Gestión de productos	Permite registrar, consultar, editar y desactivar productos.	Administrador y operador.
Gestión de categorías	Permite clasificar los productos por tipo o línea de negocio.	Administrador y operador.
Gestión de proveedores	Permite registrar proveedores asociados a los productos.	Administrador y operador.
Movimientos de inventario	Permite registrar entradas, salidas y ajustes de stock.	Administrador y operador.
Consulta de inventario	Permite revisar existencias y productos con stock bajo.	Administrador, operador y consulta.
Historial y trazabilidad	Permite consultar movimientos realizados en el inventario.	Administrador y consulta autorizada.
Logs y monitoreo	Permite registrar eventos relevantes y errores del sistema.	Administrador.

La división por módulos facilita aplicar el principio de mínimo privilegio, ya que no todos los usuarios necesitan acceder a todas las funcionalidades. De esta forma, el administrador tendrá control completo, el operador podrá gestionar productos y movimientos, y el usuario de consulta solo podrá visualizar información autorizada.

Modelo de Datos Propuesto

El modelo de datos permite identificar las entidades principales que serán almacenadas en la base de datos. Para esta aplicación se definen tablas relacionadas con usuarios, roles, productos, categorías, proveedores y movimientos de inventario, con el fin de mantener la información organizada y permitir trazabilidad sobre los cambios realizados.

Entidad	Propósito	Campos principales sugeridos
Users	Almacenar los usuarios que acceden al sistema.	id, name, email, password, role, estado
Categorías	Clasificar los productos del inventario.	id, nombre, descripción, estado
Proveedores	Registrar proveedores de productos.	id, nombre, NIT, teléfono, correo, dirección
Productos	Almacenar la información principal de cada producto.	id, nombre, código, categoría_id, proveedor_id, stock, stock_minimo, precio
Movimientos	Registrar entradas, salidas o ajustes de inventario.	id, producto_id, user_id, tipo_movimiento, cantidad, observación, fecha
Logs	Registrar eventos relevantes o errores controlados.	id, user_id, evento, detalle, fecha

Este modelo permite mantener relación entre productos, categorías, proveedores y movimientos de inventario. También permite saber qué usuario realizó una acción específica, lo cual fortalece la trazabilidad y facilita la revisión de incidentes.

Modelado de Amenazas con STRIDE

Para identificar amenazas de seguridad se utiliza la metodología STRIDE, la cual permite analizar posibles riesgos relacionados con suplantación, manipulación, repudio, divulgación de información, denegación de servicio y elevación de privilegios. Esta técnica permite relacionar cada amenaza con un componente del sistema y definir controles preventivos desde el diseño.

Categoría STRIDE	Amenaza en el sistema de inventarios	Componente afectado	Control propuesto
Spoofing / Suplantación	Un atacante intenta ingresar usando credenciales de otro usuario.	Autenticación	Uso de login seguro, hash de contraseñas y control de sesiones.
Tampering / Manipulación	Un usuario modifica cantidades de inventario sin autorización.	Movimientos de inventario	Middleware por roles, validaciones y registro del usuario que realiza la acción.
Repudiation / Repudio	Un usuario niega haber realizado una entrada o salida de inventario.	Historial de movimientos	Registro de usuario, fecha, tipo de movimiento y observación.
Information Disclosure / Divulgación	Exposición de información interna por errores técnicos visibles.	Manejo de errores	Mensajes genéricos al usuario y registro técnico en logs.

Denial of Service / Denegación de servicio	Uso excesivo o solicitudes repetidas que afectan la disponibilidad.	Aplicación web	Validaciones, control de errores y pruebas de carga básicas.
Elevation of Privilege / Elevación de privilegios	Un operador intenta acceder a funciones administrativas.	Rutas y middleware	Control de roles y restricción de rutas protegidas.

El análisis STRIDE permite anticipar riesgos que podrían afectar la confidencialidad, integridad y disponibilidad de la aplicación. A partir de estas amenazas se definen controles de seguridad que serán implementados en Laravel durante la etapa de desarrollo.

Controles de Seguridad Definidos

Con base en el análisis de amenazas, se definen los controles de seguridad que serán aplicados en el desarrollo de la aplicación. Estos controles buscan prevenir accesos indebidos, proteger la información, validar datos, mantener trazabilidad y evitar que los errores del sistema expongan información sensible.

Control de seguridad	Descripción	Amenaza que mitiga
Autenticación con Laravel Breeze	Permite implementar inicio de sesión, cierre de sesión y gestión básica de usuarios.	Suplantación de identidad.
Hash de contraseñas	Las contraseñas no se almacenan en texto plano.	Robo o exposición de credenciales.
Middleware de roles	Restringe rutas según el perfil del usuario.	Elevación de privilegios y acceso no autorizado.
Validación de formularios	Verifica que los datos ingresados cumplan reglas antes de guardarse.	Inyección, datos inválidos y manipulación.
Uso de Eloquent ORM	Evita construir consultas SQL manualmente con datos del usuario.	Inyección SQL.
Protección CSRF	Protege formularios contra solicitudes no autorizadas.	Manipulación de acciones desde sitios externos.
Manejo seguro de errores	Evita mostrar información técnica al usuario final.	Divulgación de información.
Logs de eventos	Registra acciones importantes y errores.	Repudio y falta de trazabilidad.
Control de stock mínimo	Permite detectar productos en niveles críticos.	Riesgos operativos de disponibilidad de inventario.
Separación por módulos	Divide responsabilidades y facilita mantenimiento.	Errores de diseño y exceso de privilegios.

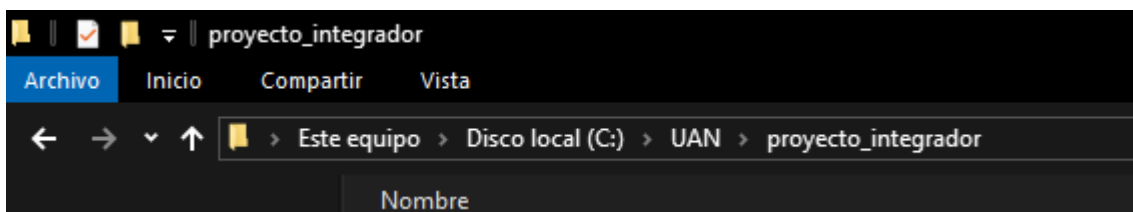
Estos controles se seleccionan porque están alineados con los riesgos identificados y con las prácticas trabajadas durante el semestre, como autenticación, autorización por roles, validación de entradas, manejo de errores, registro de eventos y pruebas de seguridad. En actividades anteriores también se trabajó el enfoque de prevenir errores comunes como SQL Injection, XSS, exposición de errores internos, ausencia de validaciones y fallas de monitoreo.

Desarrollo de Código en Laravel

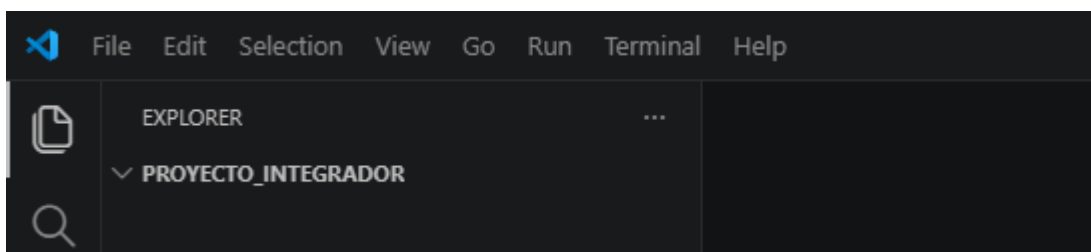
Para iniciar el entorno local de desarrollo, se abrió XAMPP y se activaron los servicios de Apache y MySQL. Apache permite ejecutar la aplicación web localmente, mientras que MySQL se utiliza como motor de base de datos para almacenar la información del sistema de inventarios.



Se creó una carpeta de trabajo llamada `proyecto_integrador`, donde se almacenó el proyecto Laravel. Esta organización permite mantener separados los archivos del proyecto final y facilita su posterior publicación en GitHub.



El proyecto fue abierto en Visual Studio Code, que se utilizó como entorno principal de desarrollo. Desde este editor se gestionaron los archivos de Laravel, las vistas Blade, los controladores, modelos, rutas, migraciones y configuraciones del sistema.



Se utilizó la terminal integrada de Visual Studio Code para ejecutar los comandos necesarios del proyecto, como la creación de la aplicación Laravel, instalación de dependencias, ejecución de migraciones, creación de controladores y levantamiento del servidor local.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\UAN\proyecto_integrador>

```

Antes de crear el proyecto Laravel, se verificó que las herramientas necesarias estuvieran instaladas correctamente. Para ello se consultaron las versiones de PHP, Composer, Node.js y NPM desde la terminal. Estas herramientas permiten ejecutar Laravel, instalar dependencias del backend y compilar los recursos del frontend.

```

PS C:\UAN\proyecto_integrador> php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies

PS C:\UAN\proyecto_integrador> composer -V
Composer version 2.9.7 2026-04-14 13:31:52
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.

PS C:\UAN\proyecto_integrador> node -v
v25.7.0

PS C:\UAN\proyecto_integrador> npm -v
11.10.1

```

Después de validar el entorno, se creó el proyecto Laravel llamado inventario-seguro. Este framework fue seleccionado porque permite organizar el código mediante el patrón MVC y facilita el uso de rutas, controladores, modelos, migraciones, middleware, validaciones y autenticación.

```

PS C:\UAN\proyecto_integrador> composer create-project laravel/laravel inventario-seguro

INFO: Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
> @php artisan migrate --graceful --ansi

INFO: Preparing database.

Creating migration table ..... 8.39ms DONE

INFO: Running migrations.

0001_01_01_000000_create_users_table ..... 10.00ms DONE
0001_01_01_000001_create_cache_table ..... 12.22ms DONE
0001_01_01_000002_create_jobs_table ..... 16.80ms DONE

```

Se verificó la versión instalada de Laravel mediante el comando `php artisan --version`. Esto confirma que el proyecto fue creado correctamente y que el framework quedó disponible para iniciar el desarrollo de la aplicación.

```

PS C:\UAN\proyecto_integrador\inventario-seguro> php artisan --version
Laravel Framework 12.61.0

```

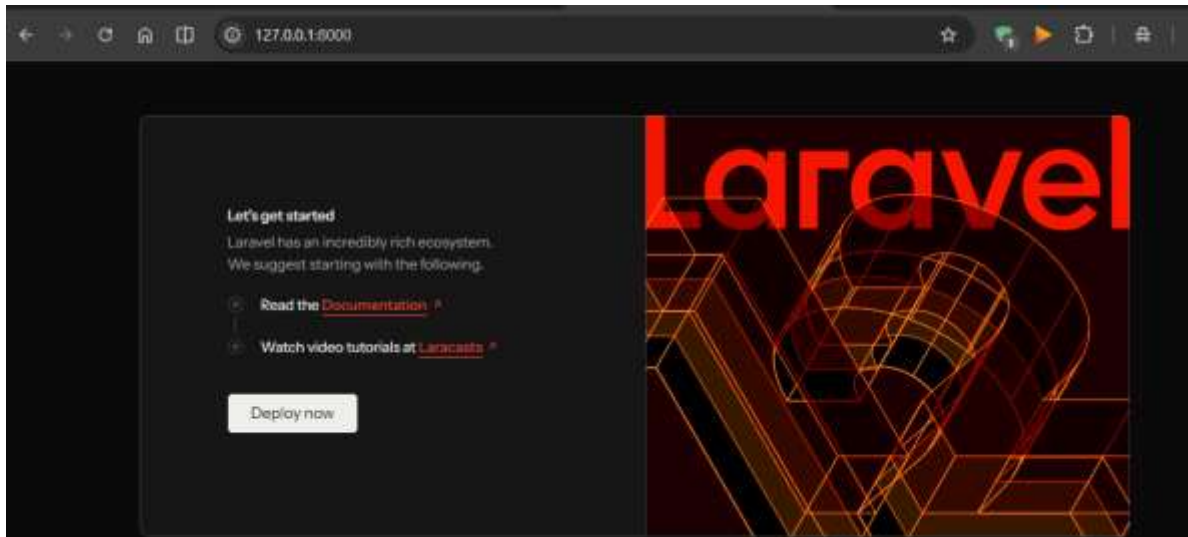
Para comprobar que la aplicación funcionaba correctamente, se ejecutó el servidor local de Laravel mediante el comando `php artisan serve`. Este comando habilita la aplicación en la dirección local `http://127.0.0.1:8000`.

```
PS C:\UAN\proyecto_integrador\inventario-seguro> php artisan serve
```

INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server

Al ingresar desde el navegador a la dirección local del proyecto, se visualizó la página inicial de Laravel. Esto permitió confirmar que la aplicación base estaba funcionando correctamente antes de iniciar la implementación de los módulos del sistema de inventarios.



Se creó en phpMyAdmin la base de datos `inventario_seguro_db`, la cual será utilizada para almacenar usuarios, roles, categorías, proveedores, productos, movimientos de inventario y demás información necesaria para el funcionamiento de la aplicación.



Luego de crear la base de datos, se configuró el archivo `.env` del proyecto para establecer la conexión con MySQL.

```
23 DB_CONNECTION=sqlite
24 # DB_HOST=127.0.0.1
25 # DB_PORT=3306
26 # DB_DATABASE=laravel
27 # DB_USERNAME=root
28 # DB_PASSWORD=
```


Allí se definieron el motor de base de datos, host, puerto, nombre de la base de datos, usuario y contraseña utilizados en el entorno local.

```
23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=inventario_seguro_db
27 DB_USERNAME=root
28 DB_PASSWORD=
```

Una vez configurada la conexión con MySQL, se ejecutaron las migraciones iniciales de Laravel. Este proceso creó las tablas base del framework, como users, cache, jobs y sessions, necesarias para la autenticación y funcionamiento interno del sistema.

```
PS C:\UAW\proyecto_integrador\inventario-seguro> php artisan migrate

INFO: Preparing database.

Creating migration table ..... 11.44ms DONE

INFO: Running migrations.

0001_01_01_000000_create_users_table ..... 52.41ms DONE
0001_01_01_000001_create_cache_table ..... 45.54ms DONE
0001_01_01_000002_create_jobs_table ..... 52.30ms DONE
```

Implementación De Autenticación con Laravel Breeze

Con la instalación de Laravel Breeze, se crea de forma rápida las pantallas de:

- Registro de usuario.
- Inicio de sesión.
- Cierre de sesión.
- Recuperación de contraseña.
- Dashboard inicial.
- Protección básica de rutas autenticadas.

Se instaló Laravel Breeze como paquete de autenticación para el proyecto. Esta herramienta permite generar una estructura inicial segura para el registro de usuarios, inicio de sesión, cierre de sesión y protección de rutas autenticadas, evitando desarrollar estos componentes desde cero.

```
PS C:\UAW\proyecto_integrador\inventario-seguro> composer require laravel/breeze --dev

laravel/breeze ..... DONE
laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE
```

Luego de instalar el paquete, se generó la estructura de autenticación utilizando Blade, el motor de plantillas de Laravel. Esta opción permite trabajar con vistas sencillas y mantener una estructura organizada para el proyecto.


```
PS C:\UAN\proyecto_integrador\inventario-seguro> php artisan breeze:install blade
> build
> vite build

vite v7.3.3 building client environment for production...
transforming...
✓ 56 modules transformed.
rendering chunks...
computing gzip size...
public/build/manifest.json      0.33 kB | gzip: 0.17 kB
public/build/assets/app-CEwZte8.css 35.31 kB | gzip: 6.63 kB
public/build/assets/app-BjMeHjpC.js 88.55 kB | gzip: 32.75 kB
✓ built in 1.53s
```

Posteriormente, se instalaron y compilaron las dependencias del frontend mediante NPM. Este paso permite que los estilos y recursos visuales de Laravel Breeze funcionen correctamente en el navegador.

```
PS C:\UAN\proyecto_integrador\inventario-seguro> npm run build

> build
> vite build

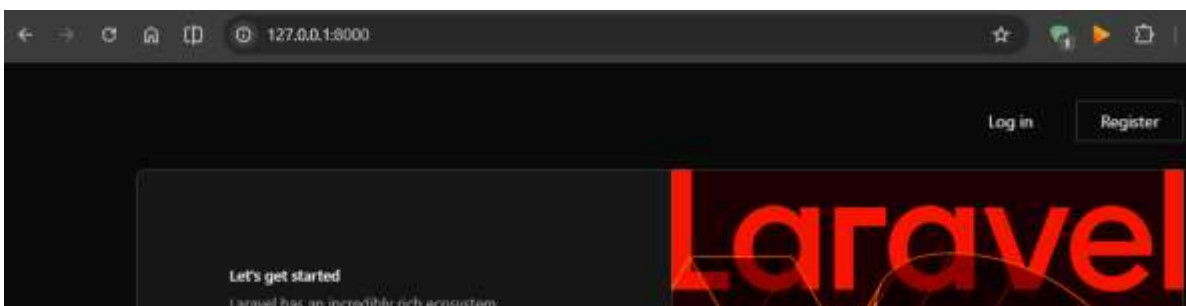
vite v7.3.3 building client environment for production...
✓ 56 modules transformed.
public/build/manifest.json      0.33 kB | gzip: 0.17 kB
public/build/assets/app-CEwZte8.css 35.31 kB | gzip: 6.63 kB
public/build/assets/app-BjMeHjpC.js 88.55 kB | gzip: 32.75 kB
✓ built in 1.46s
```

Después de instalar Laravel Breeze, se ejecutaron nuevamente las migraciones para validar que la base de datos estuviera actualizada. Este proceso asegura que las tablas necesarias para autenticación y funcionamiento interno del sistema estén correctamente creadas.

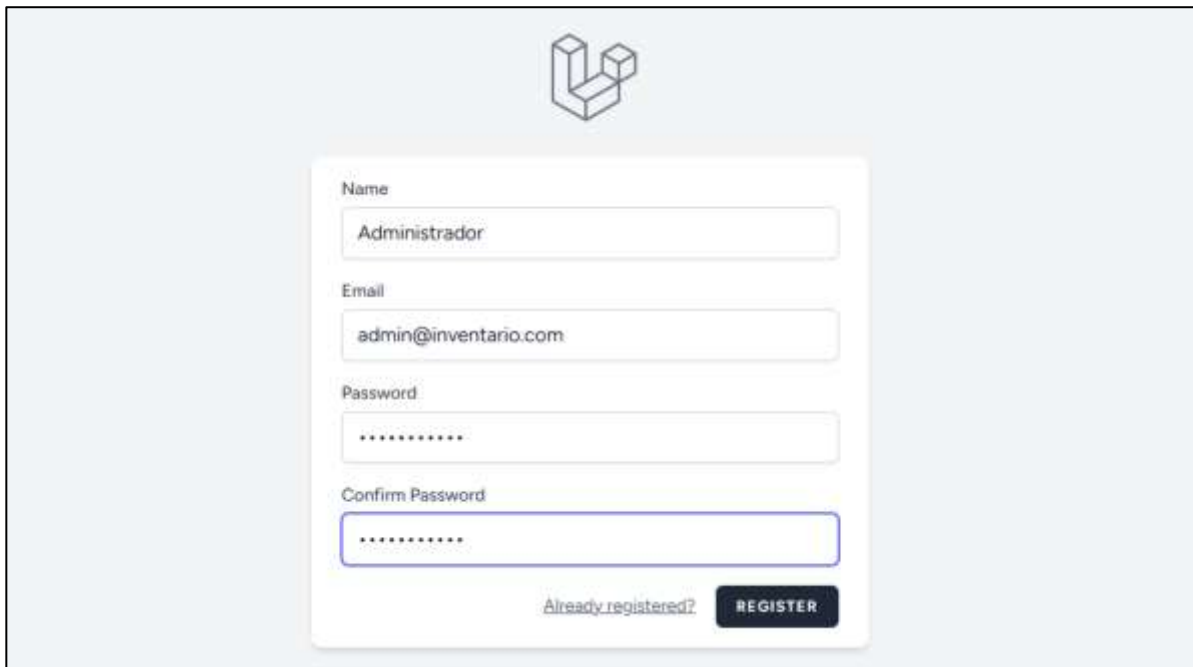
```
PS C:\UAN\proyecto_integrador\inventario-seguro> php artisan migrate

INFO Nothing to migrate.
```

Al ejecutar de nuevo el servidor aparece opciones Log in - Register

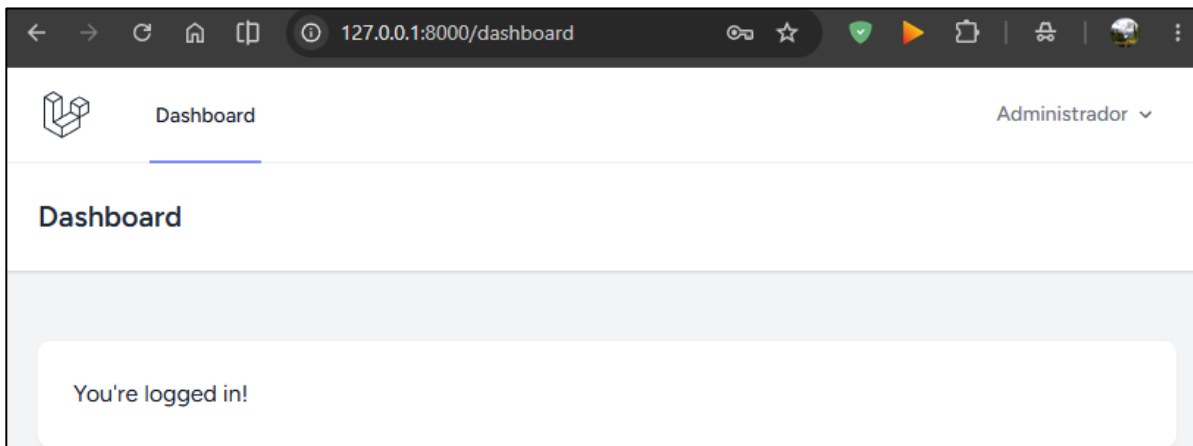


Se realizó el registro de un usuario administrador de prueba para validar el funcionamiento de la autenticación. Esta prueba permitió comprobar que el sistema permite crear usuarios y almacenar sus credenciales de forma segura mediante hash.

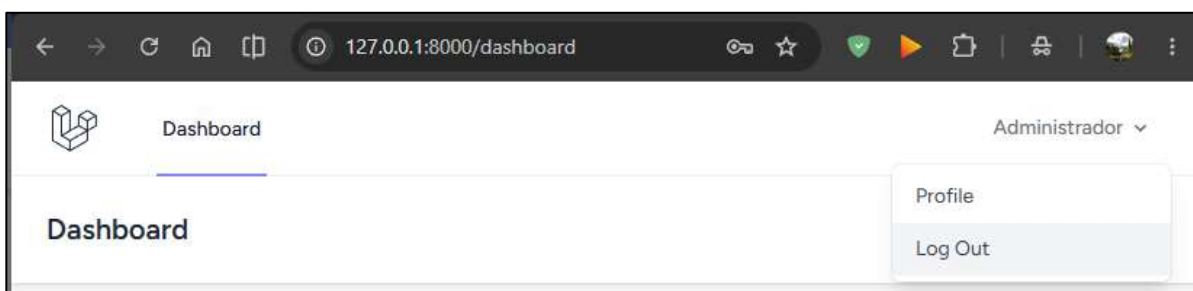


The image shows a registration form for an administrator user. At the top center is a logo consisting of three stacked cubes. Below the logo is a form with the following fields: "Name" (containing "Administrador"), "Email" (containing "admin@inventario.com"), "Password" (masked with dots), and "Confirm Password" (also masked with dots). Below the "Confirm Password" field is a link that says "Already registered?" and a dark blue button labeled "REGISTER".

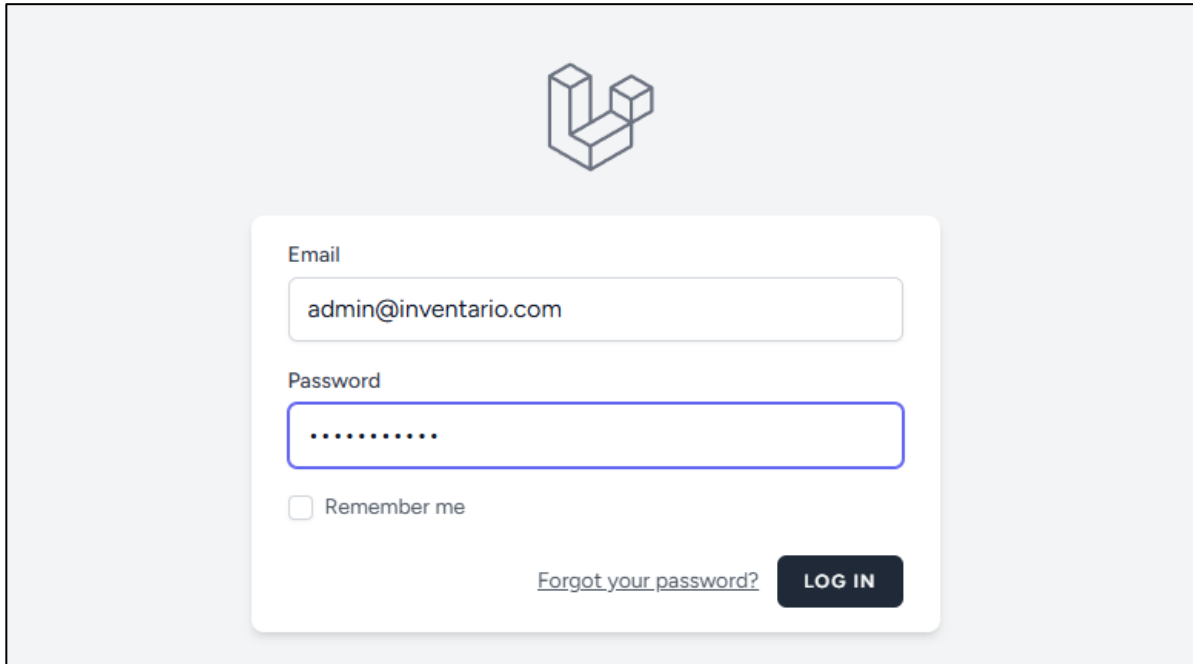
Después del registro, el sistema redirigió correctamente al Dashboard, lo cual confirma que el inicio de sesión y la gestión de sesiones funcionan adecuadamente.



También se verificó el cierre de sesión desde el menú del usuario y el posterior inicio de sesión con las credenciales registradas. Esta evidencia demuestra que Laravel Breeze administra correctamente la sesión del usuario.



Esta prueba permite confirmar que Laravel Breeze administra correctamente la sesión del usuario y protege el acceso al panel interno de la aplicación.

A screenshot of a web application's login page. At the top center is a logo consisting of three stacked cubes. Below the logo is a white login form with rounded corners. The form has two input fields: 'Email' with the value 'admin@inventario.com' and 'Password' with masked characters '.....'. Below the password field is a checkbox labeled 'Remember me'. At the bottom of the form, there is a link 'Forgot your password?' and a dark blue button labeled 'LOG IN'.

Con la implementación de Laravel Breeze, el sistema cuenta con una base inicial de autenticación segura. Esta configuración permite controlar el acceso a la aplicación, proteger rutas internas y administrar sesiones de usuario.

Desde phpMyAdmin, en la tabla de usuarios se verificó que la contraseña no se almacena en texto plano, sino mediante un hash generado por Laravel. Esto reduce el riesgo de exposición de credenciales en caso de acceso no autorizado a la base de datos.

id	name	email	email_verified_at	password	remember_token
1	Administrador	admin@inventario.com	NULL	\$2y\$12\$u6vuCC5bCn7Y5OTAjuGLLe0iZWLBkfUGuP2ZoKDrKix...	NULL

Creación de Roles de Usuario

Para implementar el control de acceso por roles, se creó una migración que agrega el campo role a la tabla users. Este campo permite diferenciar los permisos de administrador, operador y usuario de consulta.

```
php artisan make:migration add_role_to_users_table --table=users
nventario-seguro\database\migrations\2026_05_30_145952_add_role_to_users_table.php] created successfully.
```

Se crea el archivo add_role_to_users_table.php y se modifica.

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::table('users', function (Blueprint $table) {
12             $table->string('role')->default('consulta')->after('password');
13         });
14     }
15
16     public function down(): void
17     {
18         Schema::table('users', function (Blueprint $table) {
19             $table->dropColumn('role');
20         });
21     }
22 };
```

Se ejecuta la migración con el comando php artisan migrate.

```
PS C:\UAA\proyecto_integrador\inventario-seguro> php artisan migrate
INFO: Running migrations.
2026_05_30_145952_add_role_to_users_table ..... 19.71ms DONE
```

También se actualizó el modelo User app\Models\User.php para permitir el manejo del campo role dentro de la aplicación. Esto permite que Laravel pueda almacenar y consultar el rol asignado a cada usuario, facilitando posteriormente la autorización mediante middleware.

```
21     protected $fillable = [
22         'name',
23         'email',
24         'password',
25         'role',
26     ];
```

Después de agregar el campo role, se utilizó Tinker para asignar el rol de administrador al usuario principal. Esto permitió habilitar el acceso completo a las funciones de gestión del sistema.

```

● PS C:\UAN\proyecto_integrador\inventario-seguro> php artisan tinker
Psy Shell v0.12.23 (PHP 8.2.12 - cli) by Justin Hileman
New PHP manual is available (latest: 3.0.5). Update with `doc --update-manual`
> $user = App\Models\User::where('email', 'admin@inventario.com')->first();

= App\Models\User {#8133
    id: 1,
    name: "Administrador",
    email: "admin@inventario.com",
    email_verified_at: null,
    #password: "\$2y\$12\$u6vuCC5bCn7Y50TAjuGLLe0iZWLBkfUGuP2ZoKDrIX/TnvZCiLVi",
    role: "consulta",
    #remember_token: null,
    created_at: "2026-05-30 14:51:54",
    updated_at: "2026-05-30 14:51:54",
}

```

Esta configuración permite que el usuario administrador pueda acceder a las funciones de gestión y configuración del sistema de inventarios.

```

> $user->role = 'admin';

= "admin"

> $user->save();

= true

```

Se creó el middleware RoleMiddleware, encargado de validar si el usuario autenticado cuenta con el rol necesario para acceder a una ruta específica. Si el usuario no tiene permisos, el sistema bloquea el acceso mediante una respuesta 403.

```

php artisan make:middleware RoleMiddleware

inventario-seguro\app\Http\Middleware\RoleMiddleware.php] created successfully.

```

Si el usuario no ha iniciado sesión, es redirigido al login; si no cuenta con permisos suficientes, el sistema bloquea el acceso mediante un error 403.

```

 9  class RoleMiddleware
10  {
11      public function handle(Request $request, Closure $next, ...$roles): Response
12      {
13          if (!$auth()->check()) {
14              return redirect()->route('login');
15          }
16
17          if (!in_array(auth()->user()->role, $roles)) {
18              abort(403, 'No tiene permisos para acceder a esta sección.');
```

Luego de crear el middleware, se registró el alias role dentro de la configuración de Laravel. Esto permite utilizarlo directamente en las rutas protegidas del sistema para controlar el acceso según el perfil del usuario.

```
13     ->withMiddleware(function (Middleware $middleware): void {
14         $middleware->alias([
15             'role' => \App\Http\Middleware\RoleMiddleware::class,
16         ]);

```

Se definieron rutas protegidas mediante los middlewares auth y role. Esta configuración permite restringir el acceso a los módulos según el perfil del usuario, aplicando el principio de mínimo privilegio.

```
20 Route::middleware(['auth', 'role:admin'])->group(function () {
21     Route::get('/admin', function () {
22         return view('admin.dashboard');
23     })->name('admin.dashboard');
24 });
25
26 Route::middleware(['auth', 'role:admin,operador'])->group(function () {
27     Route::get('/operador', function () {
28         return view('operador.dashboard');
29     })->name('operador.dashboard');
30 });
31
32 Route::middleware(['auth', 'role:admin,operador,consulta'])->group(function () {
33     Route::get('/consulta', function () {
34         return view('consulta.dashboard');
35     })->name('consulta.dashboard');
36 });

```

Se modificó el Dashboard principal para mostrar el nombre del usuario autenticado, su rol asignado y los accesos disponibles según sus permisos. Esto permite validar visualmente que el control de acceso funciona correctamente y que cada usuario solo visualiza las opciones correspondientes a su perfil.

```
inventario-seguro > resources > views > dashboard.blade.php > x-app-layout
1 <x-app-layout>
2 <x-slot name="header">
3     <h2 class="font-semibold text-xl text-gray-800 leading-tight">
4         Dashboard Principal
5     </h2>
6 </x-slot>
7
8 <div class="py-12">
9     <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
10
11         <div class="bg-white shadow-sm sm:rounded-lg p-6 mb-6">
12             <h3 class="text-lg font-semibold">
13                 Bienvenido, {{ Auth::user()->name }}
14             </h3>
15
16             <p class="mt-2 text-gray-600">
17                 Rol asignado: <strong>{{ Auth::user()->role }}</strong>
18             </p>
19         </div>
20
21         <div class="bg-white shadow-sm sm:rounded-lg p-6">
22             <h3 class="text-lg font-semibold mb-4">
23                 Accesos disponibles
24             </h3>

```

También se creó una página personalizada para el error 403. Esta página informa al usuario que no tiene permisos para acceder a una sección determinada, sin mostrar detalles técnicos internos de la aplicación. Esto contribuye al manejo seguro de errores y evita revelar información innecesaria.

```

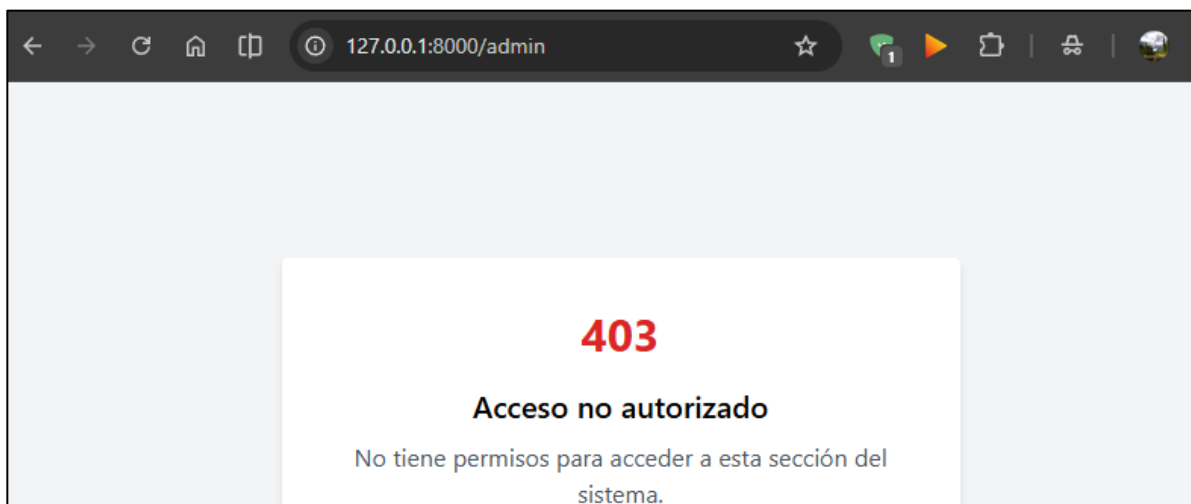
inventario-seguro > resources > views > errors > 403.blade.php > html
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <title>Acceso no autorizado</title>
6      <script src="https://cdn.tailwindcss.com"></script>
7  </head>
8  <body class="bg-gray-100 min-h-screen flex items-center justify-center">
9
10     <div class="bg-white p-8 rounded shadow-md text-center max-w-md">
11         <h1 class="text-3xl font-bold text-red-600 mb-4">403</h1>
12         <h2 class="text-xl font-semibold mb-2">Acceso no autorizado</h2>
13         <p class="text-gray-600 mb-6">
14             No tiene permisos para acceder a esta sección del sistema.
15         </p>
16
17         <a href="{{ route('dashboard') }}"
18             class="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700">
19             Volver al Dashboard
20         </a>
21     </div>
22
23 </body>
24 </html>

```

Probando los roles, quedan listas las rutas del admin.

- <http://127.0.0.1:8000/dashboard>
- <http://127.0.0.1:8000/admin>
- <http://127.0.0.1:8000/operador>
- <http://127.0.0.1:8000/consulta>

Para validar el control de acceso, se intentó ingresar al panel de administrador con un usuario sin permisos suficientes. El sistema bloqueó correctamente la solicitud y mostró una página personalizada de error 403, sin exponer información técnica interna.



Creación de Módulos Principales

Para construir la funcionalidad principal del sistema de inventarios, se crearon los módulos de categorías, proveedores y productos. Estos módulos permiten organizar los productos por tipo, asociarlos con proveedores y controlar la información básica del inventario, como código, nombre, precio, stock disponible y stock mínimo.

migraciones, modelos y controladores

```
php artisan make:model Categoria -m
inventario-seguro\app\Models\Categoria.php] created successfully.
```

```
php artisan make:model Proveedor -m
inventario-seguro\app\Models\Proveedor.php] created successfully.
```

```
php artisan make:model Producto -m
inventario-seguro\app\Models\Producto.php] created successfully.
```

Luego se crean los controladores:

```
php artisan make:controller CategoriaController --resource
inventario-seguro\app\Http\Controllers\CategoriaController.php] created successfully.
```

```
php artisan make:controller ProveedorController --resource
inventario-seguro\app\Http\Controllers\ProveedorController.php] created successfully.
```

```
php artisan make:controller ProductoController --resource
inventario-seguro\app\Http\Controllers\ProductoController.php] created successfully.
```

Después de definir las migraciones, se ejecutó el comando `php artisan migrate` para crear las tablas categorías, proveedores y productos en la base de datos MySQL. Estas tablas permiten almacenar la información principal del inventario y mantienen relaciones entre productos, categorías y proveedores.

```
PS C:\WWW\proyecto_integrador\inventario-seguro> php artisan migrate
INFO: Running migrations.
2026_05_30_162655_create_categorias_table ..... 170.72ms DONE
2026_05_30_162715_create_proveedores_table ..... 23.46ms DONE
2026_05_30_162734_create_productos_table ..... 124.13ms DONE
```


Se configuraron los modelos Categoría, Proveedor y Producto, definiendo los campos permitidos mediante \$fillable y las relaciones entre las entidades. Esto permite que Laravel gestione la información mediante Eloquent ORM, evitando construir consultas SQL manuales y reduciendo el riesgo de inyección SQL.

```
7  class Categoría extends Model
8  {
9      protected $table = 'categorias';
10
11     protected $fillable = [
12         'nombre',
13         'descripcion',
14         'estado',
15     ];
16
```

```
7  class Proveedor extends Model
8  {
9      protected $table = 'proveedores';
10
11     protected $fillable = [
12         'nombre',
13         'nit',
14         'telefono',
15         'correo',
16         'direccion',
17         'estado',
18     ];

```

```
7  class Producto extends Model
8  {
9      protected $table = 'productos';
10
11     protected $fillable = [
12         'categoria_id',
13         'proveedor_id',
14         'nombre',
15         'codigo',
16         'descripcion',
17         'stock',
18         'stock_minimo',
19         'precio',
20         'estado',
21     ];

```

Las rutas de categorías, proveedores y productos fueron protegidas mediante middleware de autenticación y roles. Los usuarios administradores y operadores pueden crear, editar y desactivar registros, mientras que el usuario de consulta solo puede visualizar el listado de productos. Esta separación aplica el principio de mínimo privilegio.

```

42 Route::middleware(['auth', 'role:admin,operador'])->group(function () {
43     Route::resource('categorias', CategoriaController::class)->except(['show']);
44     Route::resource('proveedores', ProveedorController::class)->except(['show']);
45     Route::resource('productos', ProductoController::class)->except(['show', 'index']);
46 });
47
48 Route::middleware(['auth', 'role:admin,operador,consulta'])->group(function () {
49     Route::get('/productos', [ProductoController::class, 'index']->name('productos.index'));
50 });

```

Prueba de Funcionamiento

Con la creación de los módulos de categorías, proveedores y productos, la aplicación empieza a cubrir las funcionalidades principales requeridas para la gestión de inventarios. Estos módulos incluyen validación de datos, control de acceso por roles, relaciones entre tablas y registro de eventos mediante logs. Además, se evita la eliminación directa de registros importantes, usando desactivación lógica para conservar trazabilidad dentro del sistema.

Se realizó una prueba de funcionamiento del módulo de categorías, creando una categoría llamada “Bebidas”. Esta categoría permite clasificar productos dentro del inventario y organizar la información de forma más clara.

Categorías				
Nueva categoría				
Nombre	Descripción	Estado	Acciones	
Bebidas	Bebidas en distintas presentaciones, plastico, vidrio y 3 Lt, 2 Lt, 600 ml y 300 ml	Activa	Editar	Desactivar

Se probó el módulo de proveedores mediante el registro de una empresa proveedora. Esta funcionalidad permite asociar productos con proveedores y mantener información básica de contacto.

Proveedores					
Nuevo proveedor					
Nombre	NIT	Teléfono	Correo	Estado	Acciones
Distribuciones Andinas S.A.S.	80613546842-5	6015879625	distribucionesandinas@gmail.com	Activo	Editar Desactivar

Se registró un producto de prueba asociado a una categoría y proveedor. En el listado se visualiza el código, nombre, categoría, proveedor, stock, stock mínimo, precio y estado del producto.

Productos								
Nuevo producto								
Código	Producto	Categoría	Proveedor	Stock	Stock mínimo	Precio	Estado	Acciones
GAS-001	Gaseosa 1.5L	Bebidas	Distribuciones Andinas S.A.S.	10 (Stock bajo)	15	\$4,500.00	Activo	Editar Desactivar

Movimientos de Inventario

Para complementar la gestión de productos, se implementó el módulo de movimientos de inventario. Este módulo permite registrar entradas, salidas y ajustes de stock, manteniendo trazabilidad sobre cada cambio realizado en las existencias de los productos.

Cada movimiento queda asociado a un producto y al usuario autenticado que realizó la acción. Además, el sistema guarda el stock anterior y el stock nuevo, lo cual permite revisar posteriormente qué cambios se hicieron y detectar posibles inconsistencias en el inventario.

```
php artisan make:model Movimiento -m
inventario-seguro\app\Models\Movimiento.php] created successfully.
```

```
php artisan make:controller MovimientoController --resource
inventario-seguro\app\Http\Controllers\MovimientoController.php] created successfully.
```

Se creó la tabla movimientos para registrar las operaciones realizadas sobre el inventario. Esta tabla almacena el producto afectado, el usuario que realiza la acción, el tipo de movimiento, la cantidad, el stock anterior, el stock nuevo y una observación opcional. Esta estructura permite mantener trazabilidad sobre los cambios realizados en las existencias.

```
PS C:\WWW\proyecto_integrador\inventario-seguro> php artisan migrate
INFO Running migrations.
2026_05_30_173313_create_movimientos_table ..... 279.86ms DONE
```

Se configuraron las relaciones entre los modelos Movimiento, Producto y User. De esta forma, cada movimiento queda vinculado al producto afectado y al usuario que realizó la acción. Esto fortalece la trazabilidad y permite consultar posteriormente el historial de cambios del inventario.

```

inventario-seguro > app > Models > Movimiento.php > Movimiento
7   class Movimiento extends Model
24   }
25
26   public function usuario()
27   {
28       return $this->belongsTo(User::class, 'user_id');
29   }
30 }

```

```

inventario-seguro > app > Models > Producto.php > Producto > movimientos
7   class Producto extends Model
33   public function movimientos()
34   {
35       return $this->hasMany(Movimiento::class);
36   }
37 }

```

```

inventario-seguro > app > Models > User.php > User
11  class User extends Authenticatable
51  public function movimientos()
53  {
54      return $this->hasMany(\App\Models\Movimiento::class);
55  }

```

Las rutas del módulo de movimientos fueron protegidas mediante autenticación y control de roles. Los usuarios administradores y operadores pueden registrar entradas, salidas y ajustes de inventario, mientras que el usuario de consulta solo puede visualizar el historial. Esta configuración aplica el principio de mínimo privilegio.

```

54  Route::middleware(['auth', 'role:admin,operador,consulta'])->group(function () {
55      Route::get('/movimientos', [MovimientoController::class, 'index'])
56          ->name('movimientos.index');
57  });
58
59  Route::middleware(['auth', 'role:admin,operador'])->group(function () {
60      Route::get('/movimientos/create', [MovimientoController::class, 'create'])
61          ->name('movimientos.create');
62
63      Route::post('/movimientos', [MovimientoController::class, 'store'])
64          ->name('movimientos.store');
65  });

```

En el controlador de movimientos se implementó la lógica para registrar entradas, salidas y ajustes de inventario. Antes de guardar el movimiento, el sistema valida los datos ingresados y calcula el nuevo stock del producto. También se evita que una salida deje el inventario en valores negativos, lo cual protege la integridad de la información.

```

inventario-seguio > apip > http > Controllers > MovimientoController.php > MovimientoController
11 class MovimientoController extends Controller
12 public function create()
29 }
30
31 public function store(Request $request)
32 {
33     $validated = $request->validate([
34         'producto_id' => 'required|exists:productos,id',
35         'tipo' => 'required|in:entrada,salida,ajuste',
36         'cantidad' => 'required|integer|min:0',
37         'observacion' => 'nullable|string|max:500',
38     ]);
39
40     if (in_array($validated['tipo'], ['entrada', 'salida']) && $validated['cantidad'] < 1) {
41         return back()
42             ->withErrors(['cantidad' => 'La cantidad debe ser mayor a cero para entradas y salidas.'])
43             ->withInput();
44     }
45

```

Se realizó una prueba de entrada de inventario registrando el ingreso de 20 unidades del producto seleccionado.

Nuevo movimiento de inventario

Producto
Gaseosa 1.5L - Stock actual: 10

Tipo de movimiento
Entrada

Cantidad
20

Nota: en entradas y salidas, la cantidad aumenta o disminuye el stock. En ajustes, la cantidad será el nuevo stock total del producto.

Observación
Ingreso de mercancía del proveedor.

Guardar movimiento Cancelar

Como resultado, el sistema aumentó el stock disponible y registró el movimiento en el historial, indicando usuario, cantidad, stock anterior y stock nuevo.

Historial de movimientos

Movimiento registrado correctamente.

[Nuevo movimiento](#)

Fecha	Producto	Usuario	Tipo	Cantidad	Stock anterior	Stock nuevo	Observación
30/05/2026 17:49	Gaseosa 1.5L	Administrador	Entrada	20	10	30	Ingreso de mercancía del proveedor.

Se realizó una prueba de salida de inventario registrando la disminución de 5 unidades.

Nuevo movimiento de inventario

Producto
Gaseosa 1.5L - Stock actual: 30

Tipo de movimiento
Salida

Cantidad
5

Nota: en entradas y salidas, la cantidad aumenta o disminuye el stock. En ajustes, la cantidad será el nuevo stock total del producto.

Observación
Salida por distribución a punto de venta.

Guardar movimiento Cancelar

El sistema actualizó correctamente el stock del producto y dejó evidencia del movimiento en el historial.

Historial de movimientos

Movimiento registrado correctamente.

Nuevo movimiento

Fecha	Producto	Usuario	Tipo	Cantidad	Stock anterior	Stock nuevo	Observación
30/05/2026 17:50	Gaseosa 1.5L	Administrador	Salida	5	30	25	Salida por distribución a punto de venta.
30/05/2026 17:49	Gaseosa 1.5L	Administrador	Entrada	20	10	30	Ingreso de mercancía del proveedor.

Se realizó una prueba de ajuste de inventario estableciendo el stock del producto en 12 unidades.

Nuevo movimiento de inventario

Producto
Gaseosa 1.5L - Stock actual: 25

Tipo de movimiento
Ajuste

Cantidad
12

Nota: en entradas y salidas, la cantidad aumenta o disminuye el stock. En ajustes, la cantidad será el nuevo stock total del producto.

Observación
Ajuste por conteo físico.

[Guardar movimiento](#) [Cancelar](#)

A diferencia de una entrada o salida, el ajuste define directamente el nuevo valor total del inventario.

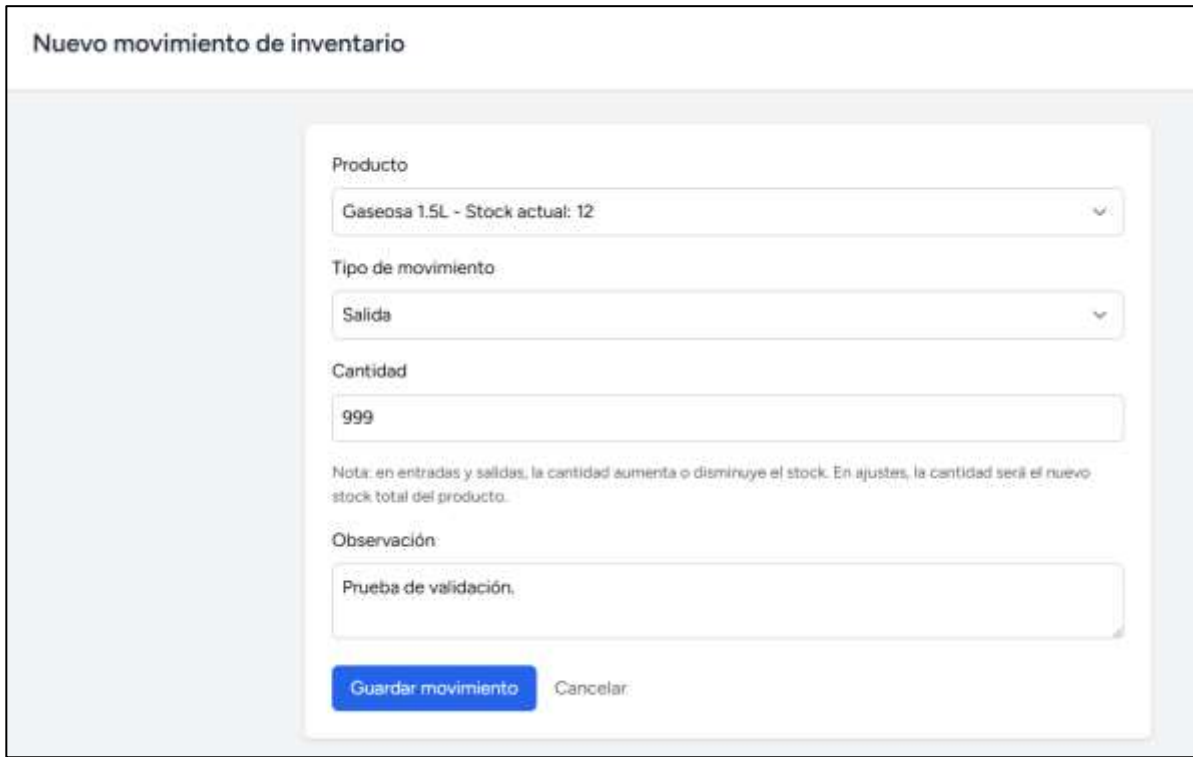
Historial de movimientos

Movimiento registrado correctamente.

[Nuevo movimiento](#)

Fecha	Producto	Usuario	Tipo	Cantidad	Stock anterior	Stock nuevo	Observación
30/05/2026 17:51	Gaseosa 1.5L	Administrador	Ajuste	12	25	12	Ajuste por conteo físico.
30/05/2026 17:50	Gaseosa 1.5L	Administrador	Salida	5	30	25	Salida por distribución a punto de venta.
30/05/2026 17:49	Gaseosa 1.5L	Administrador	Entrada	20	10	30	Ingreso de mercancía del proveedor.

Se intentó registrar una salida mayor al stock disponible para validar la integridad del inventario.



Nuevo movimiento de inventario

Producto
Gaseosa 1.5L - Stock actual: 12

Tipo de movimiento
Salida

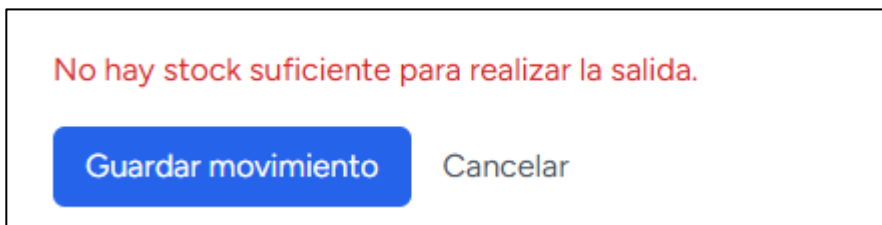
Cantidad
999

Nota: en entradas y salidas, la cantidad aumenta o disminuye el stock. En ajustes, la cantidad será el nuevo stock total del producto.

Observación:
Prueba de validación.

Guardar movimiento Cancelar

El sistema rechazó la operación y mostró un mensaje controlado indicando que no hay stock suficiente para realizar la salida.



No hay stock suficiente para realizar la salida.

Guardar movimiento Cancelar

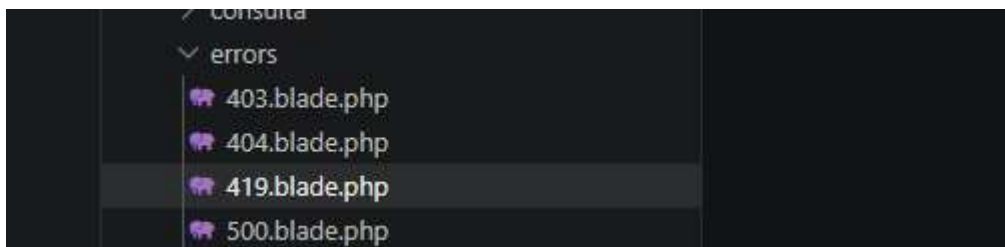
Manejo de errores, logs y Monitoreo Básico

Para fortalecer la seguridad de la aplicación, se implementaron controles de manejo de errores, registro de eventos y monitoreo básico. El objetivo es evitar que el sistema muestre información técnica sensible al usuario final, registrar acciones importantes en archivos de log y contar con una vista de monitoreo que permita revisar el estado general del inventario.

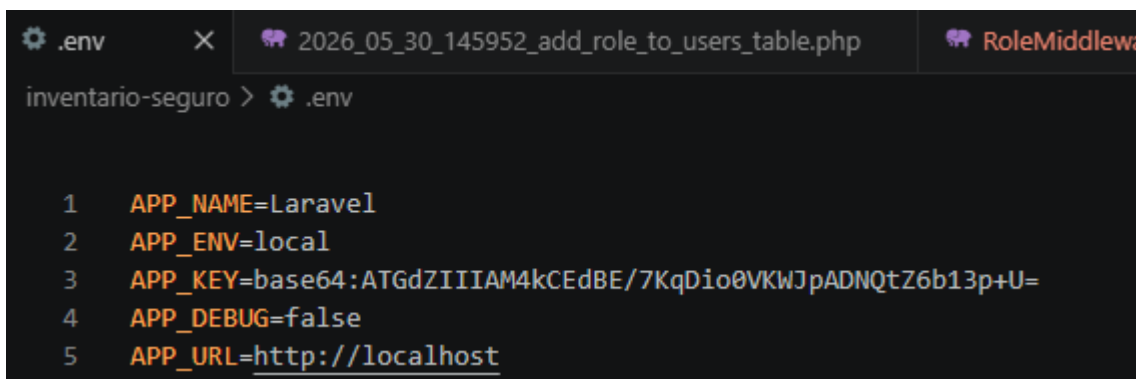
Durante esta etapa se corrigió el manejo de errores en el módulo de movimientos, se verificaron los registros generados por Laravel y se creó un panel básico de monitoreo para el administrador.

Páginas Personalizadas De Error

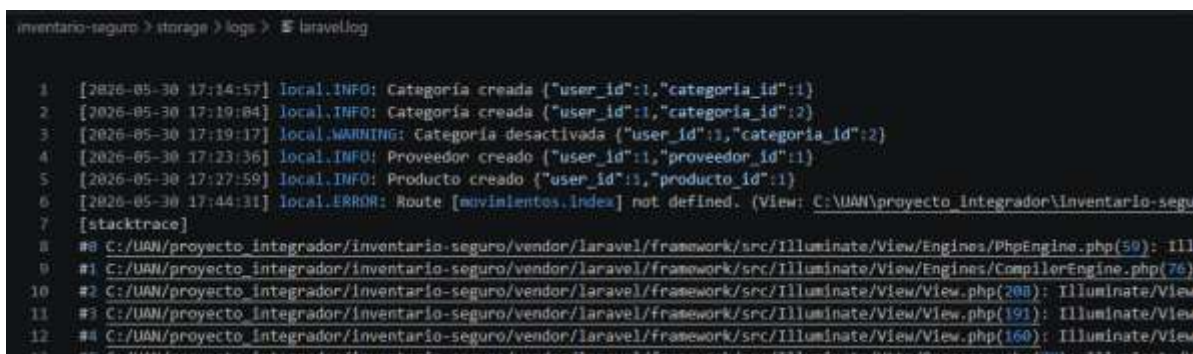
Se crearon páginas personalizadas de error para manejar respuestas como 403, 404, 419 y 500. Estas páginas permiten mostrar mensajes claros al usuario sin revelar rutas internas, archivos, controladores o líneas de código.



Se revisó la variable APP_DEBUG del archivo .env. En desarrollo puede mantenerse activa para depuración, pero en un entorno de entrega o producción debe estar en false para evitar la exposición de información técnica.

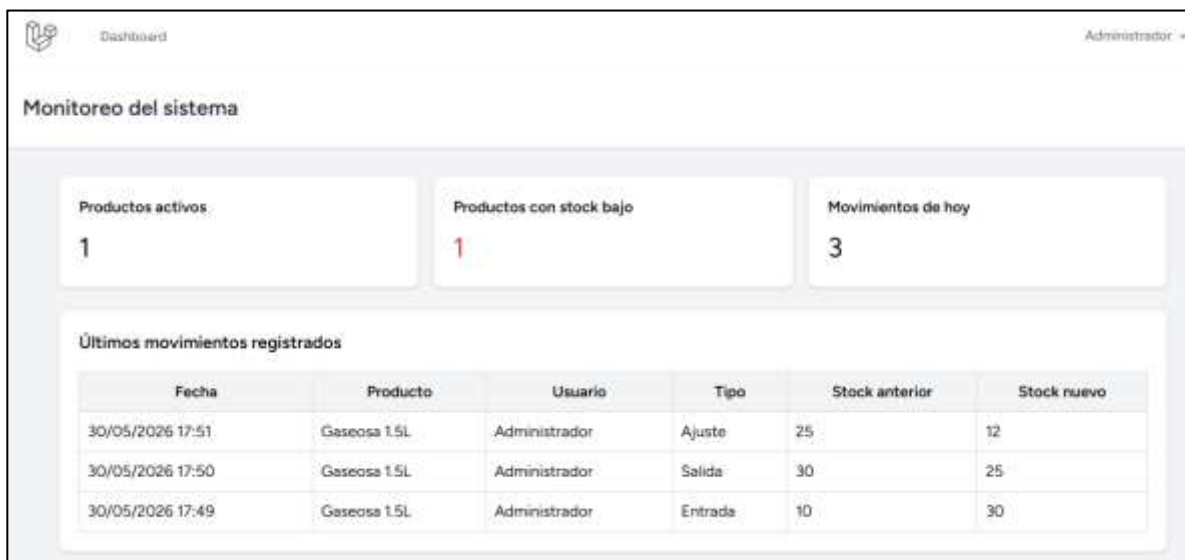


Se revisó el archivo storage/logs/laravel.log, donde Laravel registra eventos importantes del sistema. En este proyecto se generaron logs para acciones como creación de productos, actualización de registros, desactivaciones y movimientos de inventario.



Prueba de Monitoreo

Se implementó un panel básico de monitoreo para el usuario administrador. Este panel permite visualizar productos activos, productos con stock bajo, movimientos registrados en el día y últimos movimientos realizados en el inventario.



Pruebas

En esta fase se realizan pruebas funcionales, no funcionales y de seguridad sobre la aplicación web de gestión de inventarios. El objetivo es validar que los módulos desarrollados funcionen correctamente, que los usuarios solo accedan a las funciones autorizadas y que el sistema responda adecuadamente ante entradas inválidas o intentos básicos de ataque.

Las pruebas se realizan en un entorno local utilizando Laravel, MySQL, XAMPP, Visual Studio Code y navegador web.

Entorno de Pruebas

Para ejecutar las pruebas se utilizó el mismo entorno local empleado durante el desarrollo de la aplicación. Esto permite validar el comportamiento del sistema antes de una posible entrega o despliegue en un ambiente productivo.

Elemento	Herramienta utilizada
Framework	Laravel
Base de datos	MySQL
Servidor local	XAMPP / PHP Artisan Serve
IDE	Visual Studio Code
Navegador	Google Chrome
Sistema probado	Sistema Web Seguro de Gestión de Inventarios
Roles evaluados	Administrador, operador y consulta

Este entorno permite validar la autenticación, autorización por roles, formularios, registros de inventario, movimientos de stock, manejo de errores y monitoreo básico del sistema.

Casos de Prueba Funcionales

Los casos de prueba funcionales permiten comprobar que las funciones principales del sistema cumplen con lo esperado. Para esta aplicación se evaluaron acciones como inicio de sesión, creación de categorías, registro de proveedores, creación de productos y movimientos de inventario.

Código	Caso de prueba	Datos de prueba	Resultado esperado	Estado
CPF-01	Iniciar sesión con usuario válido	admin@inventario.com / contraseña correcta	El sistema permite ingresar al Dashboard	Aprobado
CPF-02	Crear categoría	Bebidas	La categoría se registra correctamente	Aprobado
CPF-03	Crear proveedor	Distribuciones Andinas S.A.S.	El proveedor queda registrado	Aprobado
CPF-04	Crear producto	Gaseosa 1.5L / GAS-001	El producto se crea y aparece en el listado	Aprobado
CPF-05	Registrar entrada de inventario	Entrada de 20 unidades	El stock aumenta correctamente	Aprobado
CPF-06	Registrar salida de inventario	Salida de 5 unidades	El stock disminuye correctamente	Aprobado
CPF-07	Registrar ajuste de inventario	Ajuste a 12 unidades	El stock queda actualizado al valor indicado	Aprobado
CPF-08	Consultar historial de movimientos	Ver módulo de movimientos	Se muestra producto, usuario, tipo, stock anterior y nuevo	Aprobado
CPF-09	Ver productos con stock bajo	Stock menor o igual al mínimo	El sistema muestra alerta de stock bajo	Aprobado
CPF-10	Acceder al panel de monitoreo	Usuario administrador	Se muestran métricas del sistema	Aprobado

Los resultados funcionales evidencian que la aplicación permite realizar las operaciones principales de inventario y mantiene trazabilidad sobre los movimientos registrados. Cada cambio en el stock queda asociado al usuario autenticado y al producto afectado.

Casos de Prueba de Autorización y Roles

También se validó el control de acceso por roles, con el fin de comprobar que cada usuario solo pueda utilizar las funciones autorizadas. Esta prueba es importante porque permite verificar el principio de mínimo privilegio aplicado durante el diseño y desarrollo.

Código	Caso de prueba	Usuario utilizado	Resultado esperado	Estado
CPA-01	Administrador accede a /admin	admin	Acceso permitido	Aprobado
CPA-02	Administrador accede a /monitoreo	admin	Acceso permitido	Aprobado
CPA-03	Operador accede a productos	operador	Acceso permitido	Aprobado
CPA-04	Operador registra movimiento	operador	Acceso permitido	Aprobado
CPA-05	Usuario consulta accede a productos	consulta	Solo puede visualizar productos	Aprobado
CPA-06	Usuario consulta intenta entrar a /admin	consulta	El sistema muestra error 403	Aprobado
CPA-07	Usuario consulta intenta entrar a /monitoreo	consulta	El sistema muestra error 403	Aprobado
CPA-08	Usuario no autenticado intenta entrar a /productos	sin sesión	El sistema redirige al login	Aprobado

Estas pruebas permitieron confirmar que el middleware de roles funciona correctamente. Los usuarios sin permisos no pueden acceder a secciones restringidas y reciben una respuesta controlada mediante la página de error 403.

Pruebas de Validación de Datos

Las pruebas de validación permiten verificar que el sistema no acepte información incompleta, inválida o que pueda afectar la integridad de los datos. Estas pruebas se realizaron sobre formularios de categorías, proveedores, productos y movimientos de inventario.

Código	Caso de prueba	Dato ingresado	Resultado esperado	Estado
CPV-01	Crear categoría sin nombre	Nombre vacío	El sistema muestra error de validación	Aprobado
CPV-02	Crear categoría duplicada	Bebidas repetida	El sistema impide el duplicado	Aprobado
CPV-03	Crear proveedor con correo inválido	correo: prueba	El sistema muestra error de validación	Aprobado
CPV-04	Crear producto sin categoría	Categoría vacía	El sistema solicita seleccionar categoría	Aprobado

CPV-05	Crear producto con código duplicado	GAS-001 repetido	El sistema impide el registro duplicado	Aprobado
CPV-06	Registrar salida mayor al stock	Salida de 999 unidades	El sistema muestra mensaje controlado	Aprobado
CPV-07	Registrar entrada con cantidad cero	Cantidad 0	El sistema no permite la operación	Aprobado

Estas pruebas demuestran que los formularios cuentan con validaciones básicas para proteger la integridad de la información. En especial, se validó que el sistema no permita salidas mayores al stock disponible, evitando inventarios negativos o inconsistentes.

Pruebas de Seguridad Básicas

Código	Prueba de seguridad	Payload o acción	Resultado esperado	Estado
CPS-01	SQL Injection en login	' OR '1'='1	El sistema no permite iniciar sesión	Aprobado
CPS-02	SQL Injection en formulario de producto	GAS-001' OR '1'='1	El dato no altera consultas ni accede a información	Aprobado
CPS-03	XSS en nombre de producto	<script>alert('xss')</script>	El script no se ejecuta en el navegador	Aprobado
CPS-04	XSS en descripción		El contenido no ejecuta código malicioso	Aprobado
CPS-05	Acceso directo sin permisos	/monitoreo con usuario consulta	El sistema muestra 403	Aprobado
CPS-06	Error de ruta inexistente	/ruta-invalida	El sistema muestra página 404 personalizada	Aprobado
CPS-07	Salida mayor al stock	Cantidad superior al inventario	El sistema muestra mensaje controlado	Aprobado
CPS-08	Protección CSRF	Envío de formulario sin token válido	Laravel rechaza la solicitud	Aprobado

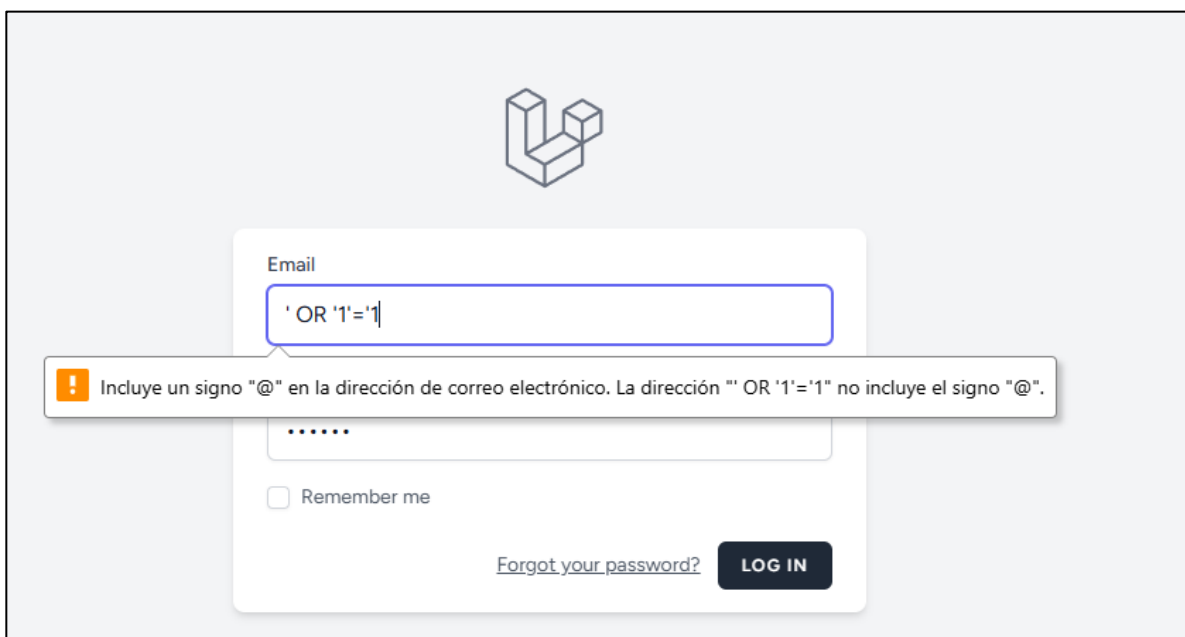
Las pruebas de seguridad realizadas permiten evidenciar que la aplicación cuenta con controles básicos contra entradas maliciosas, accesos no autorizados y exposición de errores internos. El uso de Eloquent ORM, validaciones de Laravel, middleware de roles, protección CSRF y vistas Blade contribuye a reducir riesgos comunes como inyección SQL, XSS y elevación de privilegios.

Pruebas específicas

Las pruebas específicas permiten validar controles puntuales de seguridad e integridad del sistema. En este apartado se documentan pruebas relacionadas con SQL Injection, XSS, salida mayor al stock y manejo seguro de errores.

Prueba Específica de SQL Injection

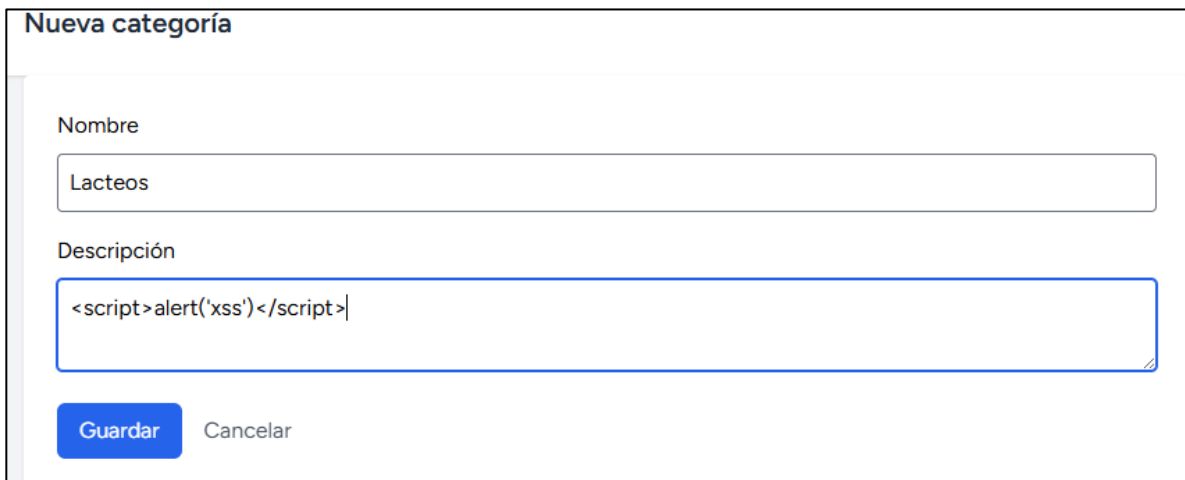
Se realizó una prueba básica de SQL Injection en el formulario de inicio de sesión, ingresando una cadena maliciosa como dato de prueba. El sistema no permitió el acceso, lo cual evidencia que la autenticación no fue vulnerada.



The screenshot shows a login interface with a logo at the top. Below the logo is a form with an "Email" label and a text input field containing the SQL injection payload: `' OR '1'='1|`. Below the email field is a password field with masked characters (dots). There is a "Remember me" checkbox and a "LOG IN" button. A red error message box is displayed over the form, stating: "Incluye un signo '@' en la dirección de correo electrónico. La dirección "' OR '1'='1" no incluye el signo '@'." Below the error message is a link that says "Forgot your password?".

Prueba Específica de XSS

Se realizó una prueba básica de XSS ingresando etiquetas JavaScript en campos de texto.



The screenshot shows a form titled "Nueva categoría". It has two text input fields: "Nombre" and "Descripción". The "Nombre" field contains the text "Lacteos". The "Descripción" field contains the XSS payload: `<script>alert('xss')</script>|`. At the bottom of the form are two buttons: "Guardar" (blue) and "Cancelar" (gray).

Al visualizar la información, el navegador no ejecutó el script, ya que Blade escapa la salida mediante {{ }}.

Categorías

Categoría creada correctamente.

[Nueva categoría](#)

Nombre	Descripción	Estado	Acciones
Bebidas	Bebidas en distintas presentaciones, plastico, vidrio y 3 Lt, 2 Lt, 600 ml y 300 ml	Activa	Editar Desactivar
Lacteos	<script>alert('xss')</script>	Activa	Editar Desactivar

Prueba de salida mayor al stock

Se realizó una prueba de integridad sobre el módulo de movimientos intentando registrar una salida mayor al stock disponible. El sistema rechazó la operación y mostró un mensaje controlado al usuario. Esta prueba confirma que la aplicación protege la integridad del inventario y evita que el stock quede en valores negativos.

Producto

Gaseosa 1.5L - Stock actual: 12

Tipo de movimiento

Salida

Cantidad

500

Nota: en entradas y salidas, la cantidad aumenta o disminuye el stock. En ajustes, la cantidad será el nuevo stock total del producto.

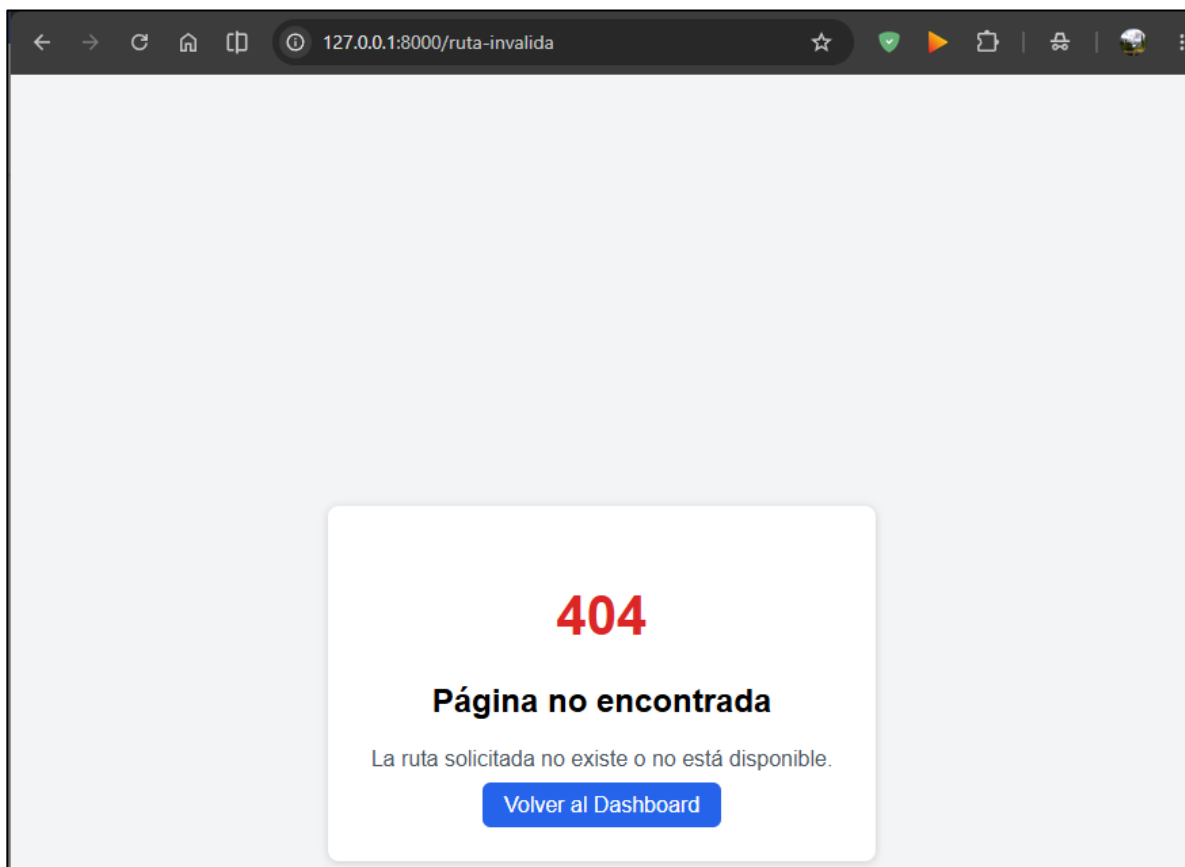
Observación

No hay stock suficiente para realizar la salida.

[Guardar movimiento](#) [Cancelar](#)

Prueba de Manejo Seguro de Errores

Se ingresó a una ruta inexistente para validar el manejo seguro de errores. El sistema mostró una página personalizada 404 en lugar de una traza técnica de Laravel.



Como resultado de estas pruebas, la aplicación demostró un comportamiento adecuado frente a escenarios funcionales y de seguridad, especialmente en la prevención de accesos no autorizados, control de stock, manejo de errores y protección contra pruebas básicas de SQL Injection y XSS.

Manejo de Incidentes

Para complementar el desarrollo seguro de la aplicación web de gestión de inventarios, se definió un proceso básico de manejo de incidentes de seguridad y fallas del sistema. Este proceso permite registrar, clasificar, priorizar, atender y documentar los incidentes detectados durante el uso o prueba de la aplicación.

El manejo de incidentes se apoya en los logs generados por Laravel, el panel de monitoreo básico implementado en la aplicación y el uso de GitHub Issues y GitHub Projects para realizar seguimiento a errores, vulnerabilidades, mejoras y tareas pendientes.

Plan de Respuesta a Incidentes

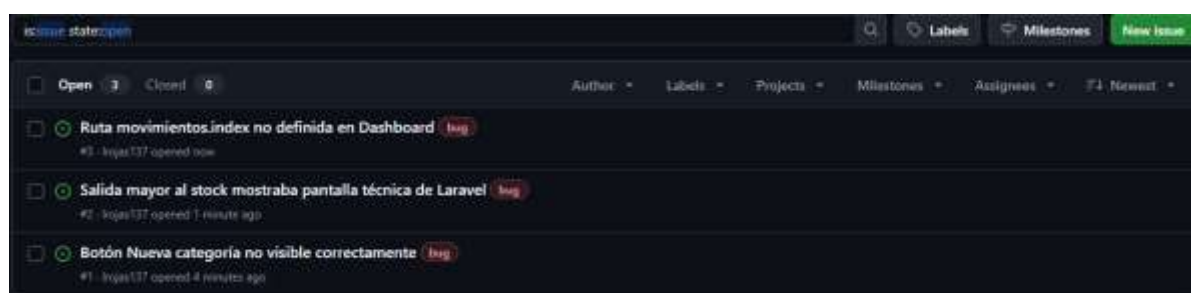
El plan de respuesta a incidentes define las etapas que se deben seguir cuando se identifica una falla, vulnerabilidad o comportamiento sospechoso en el sistema. Este flujo permite atender los incidentes de manera ordenada, evitando respuestas improvisadas y manteniendo trazabilidad sobre las acciones realizadas.

Fase	Descripción	Aplicación en el sistema de inventarios
Detección	Identificar el incidente mediante reporte de usuario, logs, pruebas o monitoreo.	Revisión de errores en formularios, accesos no autorizados, stock inconsistente o errores de Laravel.
Registro	Documentar el incidente en GitHub Issues.	Crear un issue con descripción, módulo afectado, evidencia y prioridad.
Clasificación	Determinar si es bug, incidente de seguridad, mejora o tarea.	Usar labels como bug, seguridad, mejora, prioridad alta.
Contención	Aplicar una acción temporal para evitar mayor impacto.	Bloquear acceso, desactivar una función, evitar registros incorrectos o corregir validación.
Análisis	Revisar causa raíz del problema.	Verificar rutas, controladores, logs, permisos, validaciones o base de datos.
Corrección	Implementar solución en el código o configuración.	Ajustar controlador, vista, middleware, validación o migración.
Pruebas	Validar que la solución funcione y no genere nuevos errores.	Repetir la prueba del incidente y comprobar resultado esperado.
Cierre	Documentar la solución y cerrar el issue.	Registrar evidencia final y lecciones aprendidas.

Este flujo permite controlar el ciclo completo del incidente desde su detección hasta su cierre. Además, facilita la mejora continua porque cada caso atendido deja una evidencia que puede consultarse posteriormente.

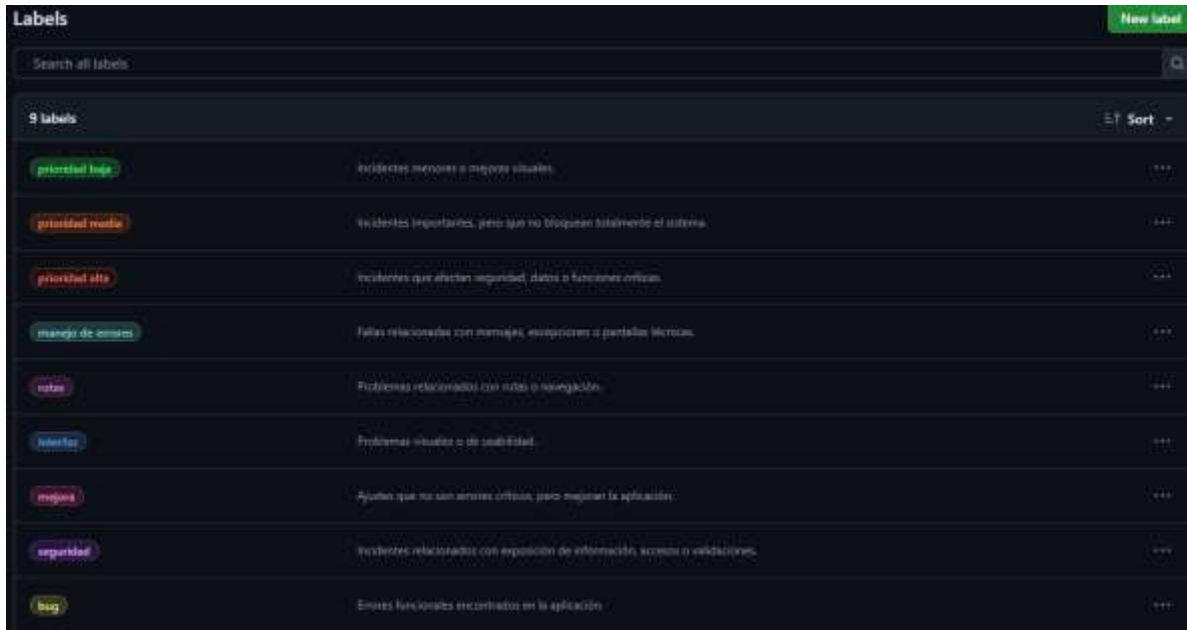
Registro de Incidentes en GitHub Issues

Se utilizó GitHub Issues para registrar incidentes reales identificados durante el desarrollo y las pruebas del sistema. Cada issue documenta el problema encontrado, módulo afectado, impacto, evidencia y solución aplicada.



Labels en GitHub

Se configuraron etiquetas en GitHub para clasificar los incidentes según tipo, prioridad y área afectada. Esto permite organizar los reportes y priorizar los casos más importantes.

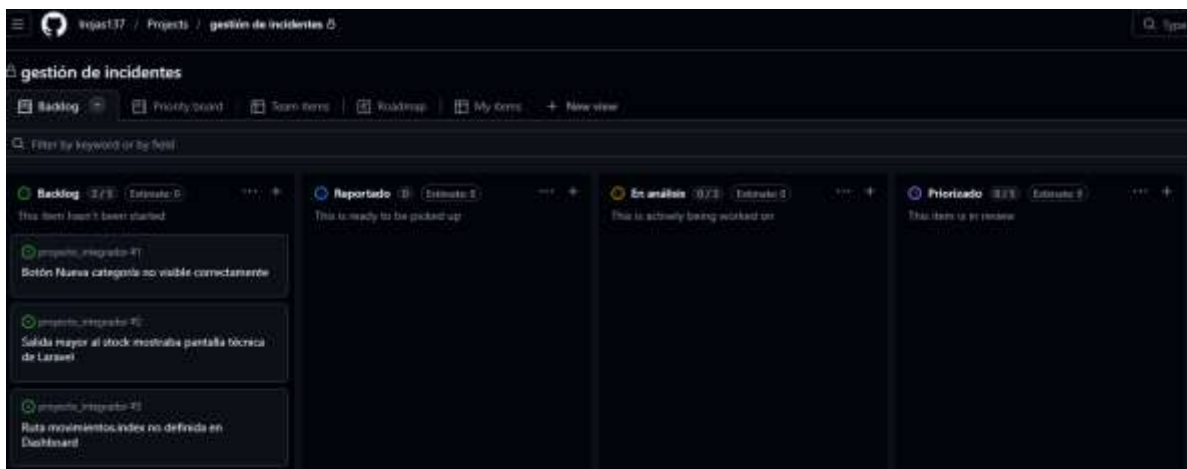


La clasificación por labels facilita identificar rápidamente qué incidentes requieren atención prioritaria y cuáles pueden resolverse como mejoras o ajustes menores.

Workflow de Gestión de Incidentes

Se configuró un tablero en GitHub Projects para gestionar el flujo de trabajo de los incidentes. El tablero permite visualizar el estado de cada caso desde que es reportado hasta que se corrige, prueba, cierra y documenta como lección aprendida.

El flujo utilizado fue: Reportado, En análisis, Priorizado, Asignado, En desarrollo, En pruebas, Cerrado y Lecciones aprendidas. Esta estructura facilita el seguimiento del trabajo y permite que cada incidente tenga un estado claro dentro del proceso.



Bitácora de Incidentes y Soluciones

Como parte de la trazabilidad del proceso, se elaboró una bitácora de incidentes con los casos identificados durante el desarrollo y prueba de la aplicación. Esta bitácora resume el tipo de incidente, prioridad, estado y solución aplicada.

Incidente	Tipo	Prioridad	Estado	Solución aplicada
Botón Nueva categoría no visible correctamente	Bug / Interfaz	Baja	Cerrado	Se ajustó el estilo del botón para mejorar contraste y visibilidad.
Salida mayor al stock mostraba error técnico	Seguridad / Manejo de errores	Alta	Cerrado	Se reemplazó la excepción general por una validación controlada.
Ruta movimientos.index no definida	Bug / Rutas	Media	Cerrado	Se guardó correctamente routes/web.php y se validó la ruta con Artisan.
Usuario sin permisos intenta acceder a monitoreo	Seguridad / Autorización	Alta	Cerrado	Se protegió la ruta con middleware role:admin.
Producto con stock bajo	Alerta operativa	Media	En seguimiento	Se visualiza alerta en productos y panel de monitoreo.

La bitácora permite documentar los incidentes atendidos y las acciones aplicadas para corregirlos. También funciona como evidencia de mejora continua, ya que permite revisar problemas recurrentes y fortalecer controles en futuras versiones.

Monitoreo y Detección de Amenazas

Para apoyar la detección de incidentes, la aplicación cuenta con registros en logs y un panel básico de monitoreo. Estos elementos permiten revisar eventos relevantes, productos con stock bajo y movimientos recientes del inventario.

Elemento monitoreado	Fuente de información	Utilidad
Productos activos	Panel de monitoreo	Permite conocer el estado general del inventario.
Productos con stock bajo	Panel de monitoreo / listado de productos	Ayuda a detectar riesgos operativos de inventario.
Movimientos del día	Panel de monitoreo	Permite identificar actividad reciente sobre el inventario.
Últimos movimientos	Tabla movimientos	Permite revisar usuario, producto, tipo de movimiento y stock actualizado.

Errores del sistema	storage/logs/laravel.log	Facilita revisar fallas, excepciones o comportamientos anómalos.
Accesos no autorizados	Página 403 / middleware de roles	Permite evidenciar intentos de acceso sin permisos.

El monitoreo implementado permite detectar fallas operativas y posibles eventos de seguridad. Aunque es un monitoreo básico, aporta visibilidad sobre el comportamiento del sistema y facilita la identificación temprana de incidentes.

Procedimiento para Manejar Vulnerabilidades y Exploits

Además de los errores funcionales, se definió un procedimiento para atender vulnerabilidades o intentos de explotación identificados durante las pruebas o el uso de la aplicación. Esto permite actuar de manera ordenada ante riesgos como SQL Injection, XSS, exposición de errores o accesos indebidos.

Vulnerabilidad o evento	Procedimiento de atención
SQL Injection	Registrar el hallazgo, revisar formularios afectados, confirmar uso de Eloquent o consultas parametrizadas, probar nuevamente y cerrar el issue.
XSS	Identificar campo afectado, validar entrada, verificar uso de {{ }} en Blade, probar payload malicioso y documentar resultado.
Acceso no autorizado	Revisar ruta afectada, validar middleware auth y role, probar con usuarios de distintos perfiles y registrar evidencia.
Exposición de errores internos	Revisar APP_DEBUG, crear página de error personalizada, controlar excepciones y validar que no se muestren trazas técnicas.
Stock inconsistente	Revisar movimientos, validar reglas de salida, confirmar transacciones de base de datos y registrar corrección.
Dependencias vulnerables	Revisar paquetes instalados, actualizar dependencias cuando aplique y documentar cambios.

Este procedimiento permite que las vulnerabilidades sean tratadas como incidentes gestionables, con registro, análisis, corrección, prueba y cierre. De esta manera, se fortalece la seguridad del ciclo de vida de la aplicación.

Métricas de Incidentes

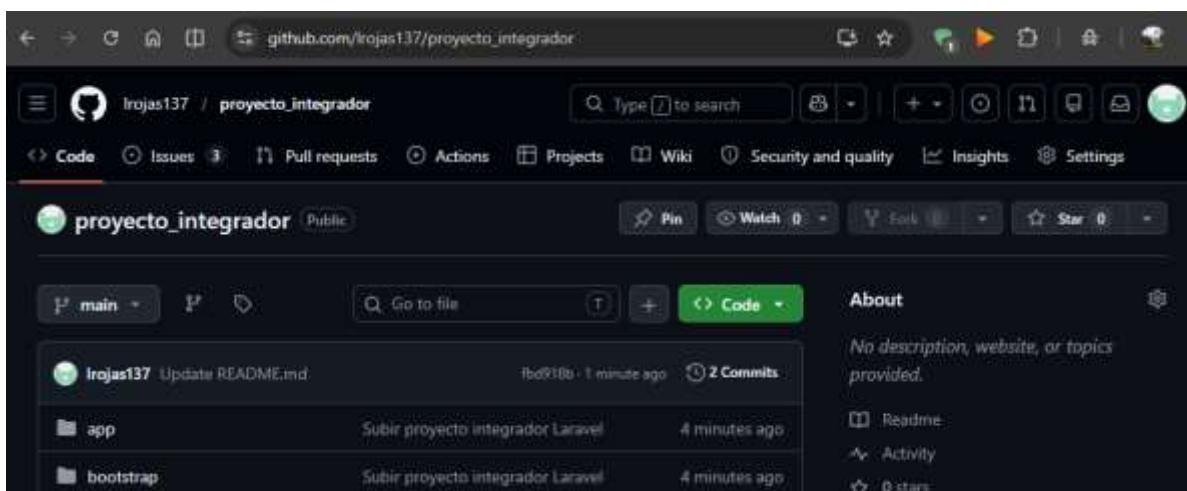
Para evaluar el proceso de gestión de incidentes se definieron métricas básicas. Estas métricas permiten conocer el estado de los incidentes, identificar tendencias y proponer acciones de mejora.

Métrica	Resultado inicial
Total de incidentes registrados	5
Incidentes cerrados	4
Incidentes en seguimiento	1
Tipo más frecuente	Bug / Manejo de errores
Módulos con más reportes	Categorías, Movimientos y Dashboard
Acción de mejora propuesta	Mejorar validaciones, estilos visuales y revisión de rutas protegidas.

Estas métricas permiten observar que la mayoría de incidentes fueron corregidos durante el desarrollo. También evidencian que los problemas más relevantes estuvieron relacionados con manejo de errores, rutas y validaciones, por lo cual se definieron acciones de mejora para futuras versiones.

Proyecto en GitHub

El código fuente de la aplicación fue publicado en un repositorio público de GitHub con el fin de entregar la evidencia del desarrollo realizado. En el repositorio se incluyeron los archivos principales del proyecto Laravel, migraciones, modelos, controladores, vistas, rutas, middleware, pruebas y documentación mediante el archivo README.md.



Para proteger información sensible, no se subió el archivo .env, ni carpetas generadas localmente como vendor y node_modules. Estas dependencias pueden instalarse nuevamente mediante los comandos composer install y npm install, siguiendo las instrucciones documentadas en el README del proyecto.

Link del repositorio: https://github.com/Irojas137/proyecto_integrador

Conclusión

El desarrollo del sistema web seguro de gestión de inventarios permitió aplicar de manera práctica los conceptos vistos durante en la asignatura de Desarrollo de Aplicaciones Seguras. A través de Laravel se implementó una aplicación organizada por módulos, con autenticación, roles, validaciones, control de rutas, manejo de errores y trazabilidad de movimientos.

Las pruebas realizadas permitieron validar el funcionamiento del sistema y comprobar controles importantes como la restricción de acceso por roles, la protección ante entradas inválidas, el bloqueo de salidas mayores al stock disponible y el manejo controlado de errores. Este proceso refuerza la importancia de probar la aplicación antes de entregarla, ya que ayuda a detectar fallas reales y corregirlas oportunamente.

Finalmente, la gestión de incidentes mediante GitHub Issues y Projects permitió documentar problemas encontrados durante el desarrollo, clasificarlos, priorizarlos y registrar sus soluciones. Esto aporta trazabilidad y mejora continua, demostrando que una aplicación segura no solo depende del código, sino también de la forma en que se administra, monitorea y mejora durante su ciclo de vida. Este enfoque coincide con lo trabajado previamente en actividades de roles en Laravel, pruebas y gestión de incidentes.

Resultado de aprendizaje

Se fortaleció la capacidad para analizar requerimientos, diseñar una arquitectura segura, implementar controles de acceso, validar entradas, manejar errores, registrar eventos, realizar pruebas y gestionar incidentes en una aplicación web.

El proyecto permitió comprender que la seguridad no debe aplicarse únicamente al final del desarrollo, sino desde las primeras etapas del ciclo de vida del software. Además, se evidenció la importancia de documentar los hallazgos, probar los controles implementados y mantener trazabilidad sobre los cambios e incidentes detectados durante la construcción de la aplicación.

Referencias

- Apache Friends. (s.f.). *XAMPP Installers and Downloads for Apache Friends*. Obtenido de Apache Friends: <https://www.apachefriends.org/es/index.html>
- Composer. (s.f.). *Composer: A Dependency Manager for PHP*. Obtenido de <https://getcomposer.org/>
- GitHub Docs. (s.f.). *About issues*. Obtenido de GitHub: <https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/about-issues>
- GitHub Docs. (s.f.). *About Projects*. Obtenido de GitHub: <https://docs.github.com/en/issues/using-labels-and-milestones-to-track-work/managing-labels>
- GitHub Docs. (s.f.). *Managing labels*. Obtenido de GitHub: <https://docs.github.com/en/issues/using-labels-and-milestones-to-track-work/managing-labels>
- Laravel. (s.f.). *Laravel Documentation*. Obtenido de <https://laravel.com/docs/12.x>
- OWASP Foundation. (s.f.). *OWASP Top 10*. Obtenido de <https://owasp.org/www-project-top-ten/>
- PHP. (s.f.). *PHP Documentation*. Obtenido de <https://www.php.net/docs.php>
- Project Management Institute. (2021). *A guide to the Project Management Body of Knowledge (PMBOK Guide)*. Obtenido de <https://www.pmi.org/standards/pmbok>